

LIST OF PRACTICALS

Sr. No	Title of the practical	Page No.	Date of performance	Date of submission	Assessment Marks (50)	Dated sign. of teacher	Remarks (if any)
1.	Install and configure Linux (or alike) operating system.						
2.	Execute general purpose commands date, time, cal, clear, banner, tty, script, man.						
3.	Work with multiple linux terminals and basic commands : who, who am I, login, passwd, su, pwd.						
4.	a) Use Operating services (Editor, GUI, File handling) b) Run commands to start, stop, and restart the specified services in Linux.						
5.	Execute process commands – ps, wait, sleep, exit, kill.						
6.	Execute file and directory manipulation commands – ls, rm, mv, cp, join, split, cat (file saving and reduction operator), head, tail, touch.						
7.	Execute file and directory manipulation commands – diff, comm, pr, chmod, mkdir, rmdir, cd, pwd, dir, cmp. (Use wild card character).						

8.	Execute text processing tr, wc, cut, paste, spell, sort, grep, more.						
9.	Use vi editor and perform all the editor commands.						
10.	Write and execute Shell Script by using following Control Statements features – “if” statement.						
11.	Write and execute Shell Script by using following Control statements features – “for” statement, exit, break, continue.						
12.	Write Shell Scripts to find out whether – Given file exists ?						
13.	Write Shell Script to find out whether – File has read, write, and execute permissions?						
14.	Write a program to calculate total waiting and turnaround time of n processes with First Come First Serve CPU scheduling algorithm.						
15.	Write a ‘C’ program to implement FIFO page replacement Algorithm.						

Practical 1 : Install and configure Linux (or alike) operating system.

PRACTICAL SIGNIFICANCE :

Linux is the base of many of open source operating system designed to replace Windows and Mac OS .It is free to download and install on any computer because it is open source. There are a variety of different versions or distributions or available developed by different by different groups. To prevent hacking attempts, Many organizations keep their LINUX operating system private, many others make their variations of Linux available publicly So the whole can benefit and large.

MINIMUM THEORETICAL BACKGROUND :

STEPS :

- 1) Download the ISO file.
- 2) Boot your system with Bootable DVD/USB drive. To start the installation click on “Install Ubuntu” .
- 3) Check Install prerequisite.
- 4) Select the Installation Type.
- 5) Select your respective Time Zone.
- 6) Select your respective Keyboard Layout.
- 7) Set the Hostname of your system and User credentials that will be used after installation. Installation has started. Once the installation is completed, it will ask to restart the Machine. Click on “Restart Now”.
- 8) Login Screen after reboot. Use the same user and its credentials that we have set during the installation. We will get below screen after entering the credentials. Ubuntu Installation is Completed Now.

Similarly any open source installation shall be considered.

PROGRAM CODE :

- 1) Install and configure Linux (or similar) operating system on your computer. Write down the steps for the same.

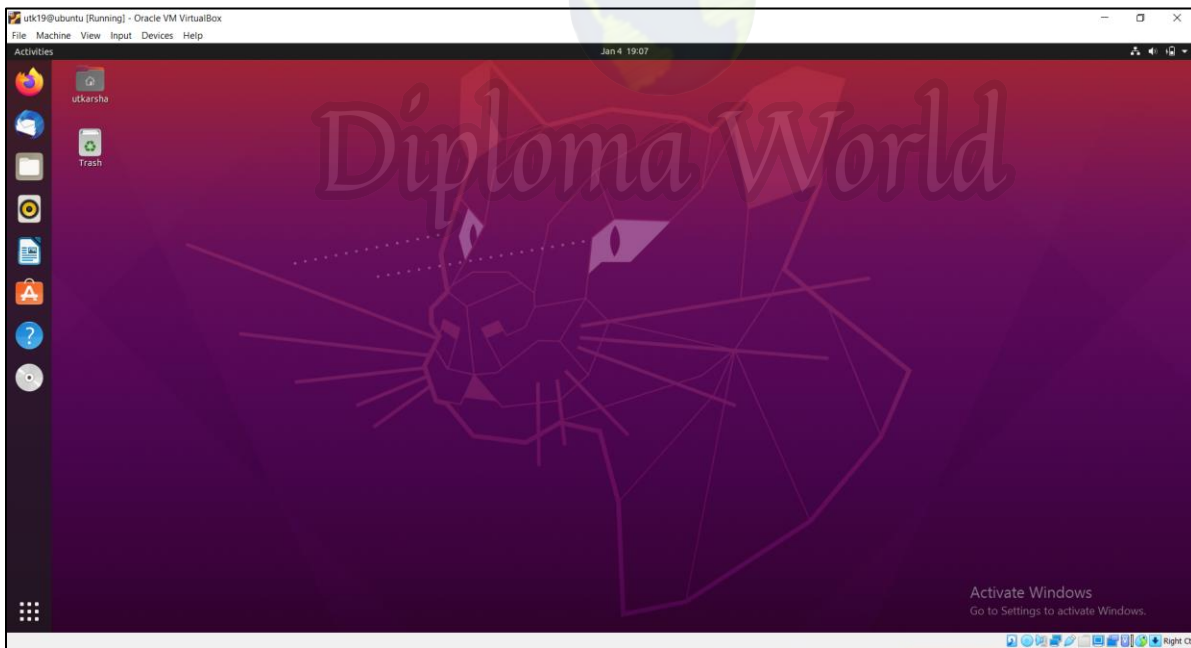
Ans.

Steps :

- 1) Download the Linux distribution of your choice.
- 2) Boot into the Live CD or Live USB.
- 3) Try out the Linux distribution before installing.
- 4) Start the installation process.
- 5) Create a username and password.
- 6) Set up the partition.
- 7) Boot into Linux.
- 8) Check your Hardware.
- 9) Start using Linux.

RESULT :

We successfully installed Linux Operating System (Ubuntu).



PRACTICAL RELATED QUESTIONS :

- 1) What are different versions of Linux Operating System?

Ans.

- Debian

- Gentoo
- Ubuntu
- Linux Mint
- Red Hat Enterprise Linux
- CentOS
- Fedora
- Kali Linux

2) Enlist the steps for booting the operating system.

Ans. Booting Process Steps :

- BIOS and setup program
- The POST (Power-On-Self-Test)
- The OS (Operating System) Loads
- System Configuration
- System Utility Loads
- Users Authentication

3) State names of latest multi-user operating system and its advantages.

Ans. Latest multi-user operating system :

- Windows NT
- Windows 2000
- Windows XP
- Windows Vista
- Mac OS X
- Linux
- FreeBSD
- NetBSD

Advantages :

- 1) You would be able to run different programs on their native OS.
- 2) You would have a great deal of versatility in what you can do with your computer.
(eg. Gaming and software development, Windows and Linux software development on the same system, etc)
- 3) You can also do things in one OS that you wouldn't be able to do in your main OS.
(Linux/FreeBSD could give you more control than Win10)

- 4) If you use your computer for work, then you can also use it for separate work and leisure.

EXERCISE :

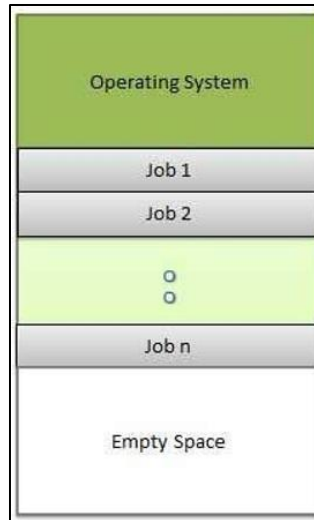
- 1) Differentiate between Command Line OS and GUI OS by giving example.

Ans.

CLI	GUI
CLI is difficult to use.	GUI is easy to use.
It consumes more memory.	It consumes less memory.
High precision is obtained.	Low precision is obtained.
Faster than GUI.	Slower than CLI.
Needs only keyboard.	Needs keyboard as well as mouse.
Input is entered only on command prompt.	Input can be entered anywhere on the screen.
No menus provided.	Menus are provided.
No use of graphics.	Uses Graphics.
Does not use any pointing devices.	It uses pointing devices for selecting or choosing options.
Examples: Microsoft Windows, DOS Shell, Mouse Systems PowerPanel	Examples: Microsoft Windows, macOS, Ubuntu Unity, Android, Apple's iOS, BlackBerry OS, Windows 10 Mobile, Palm OS-WebOS, and Firefox OS for smartphones.

- 2) Draw the diagram of multiprogramming system and state concept of it.

Ans. Sharing the processor, when two or more programs reside in memory at the same time, is referred as multiprogramming. Multiprogramming assumes a single shared processor. Multiprogramming increases CPU utilization by organizing jobs so that the CPU always has one to execute.



3) Which are the extra facilities provided by Unix other than windows OS?

Ans.

- Unix is more flexible and can be installed on many different types of machines, including main-frame computers, supercomputers and micro-computers.
- Unix is more stable and does not go down as often as Windows does, therefore requires less administration and maintenance.
- Unix has greater built-in security and permissions features than Windows.
- Unix possesses much greater processing power than Windows.

4) In list four features of the following operating system.

Ans.

a) Windows 98

1. Integrated Microsoft Internet Explorer in Windows Explorer and Help System.
2. Quick Launch tool bar.
3. Windows 98 came up with a tool called System File Checker that scans and restore corrupt or missing system files.
4. Extended support for the connection to networks.

b) Windows 2000

1. Support for FAT16, FAT32, and NTFS.
2. Increased uptime of the system and significantly fewer OS reboot scenarios.
3. Windows Installer tracks applications and recognizes and replaces missing components.

4. Protects memory of individual apps and processes to avoid a single app bringing the system down.
- c) Windows XP
1. Reliability [Home and Professional] Configured on the reliability-proven kernel of Windows 2000.
 2. Faster response even in processing multi-tasks [Home and Professional].
 3. Retention of data security and protection of user privacy.
 4. Multi-task [Home and Professional] Operate multiple applications at the same time.

CONCLUSION : We have successfully completed installation of Linux operating system. In this practical we have studied the versions, booting process, etc. of operating system.

Practical 2 : Execute general purpose commands date, time, cal, clear, banner , tty, script, man

PRACTICAL SIGNIFICANCE :

General purpose commands are inbuilt programs that can be invoked in multiple ways. These commands work interactively from a terminal. A terminal that provides a command line interface using shell program.

MINIMUM THEORETICAL BACKGROUND :

Sr. No.	Commands	Meaning
1	\$date	It displays system date and time
2	\$cal	Displays calendar for current month.
3	\$clear	Clears the screen
4	\$banner OSY	Displays an argument string as a poster with maximum of ten characters per line
5	\$tty	Name of terminal
6	\$script	Record login session
7	\$man	It gives manual help for any command.

Specifying the date format

(+ is used to concatenate date command with descriptor)

Descriptor	Example	Meaning	Example
%y	\$date "+%y"	Year(2 Digits)	19
%b	\$date "+%b"	Month abbreviation	Jan
%m	\$date "+%m"	Month digit	01
%d	\$date "+%d/%m/%y"	Day of month	01 to 31
%j	\$date "+Today is %jth day of year"	Day of year	1 to 366

PROGRAM CODE :

- 1) Execute general purpose commands date, time, cal, clear, banner , tty, script, man.

Ans.

- 1) date

```
pranavvibhute@panu-VirtualBox:~$ date
Tuesday 05 January 2021 04:02:40 PM IST
pranavvibhute@panu-VirtualBox:~$
```



```
pranavvibhute@panu-VirtualBox:~$ date "+%y"
21
pranavvibhute@panu-VirtualBox:~$ date "+%b"
Jan
pranavvibhute@panu-VirtualBox:~$ date "+%m"
01
pranavvibhute@panu-VirtualBox:~$ date "+%d"
05
pranavvibhute@panu-VirtualBox:~$ date "+%j"
005
pranavvibhute@panu-VirtualBox:~$ date "+%d-%b-%y"
05-Jan-21
pranavvibhute@panu-VirtualBox:~$
```

- 2) time

```
pranavv@pranavv:~$ time

real    0m0.000s
user    0m0.000s
sys     0m0.000s
pranavv@pranavv:~$
```

- 3) cal

```
pranavv@pranavv:~$ cal
January 2021
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
pranavv@pranavv:~$
```

4) clear

```
pranavv@pranavv:~$ cal
January 2021
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
pranavv@pranavv:~$
```

```
pranavv@pranavv:~$
```

5) banner

```
pranavv@pranavv:~$ banner PRANAV
#####  #####  #  #  #  #  #  #
#  #  #  #  #  #  #  #  #  #  #
#  #  #  #  #  #  #  #  #  #
#####  #####  #  #  #  #  #  #
#  #  #  #####  #  #  #####  #  #
#  #  #  #  #  #  #  #  #  #
#  #  #  #  #  #  #  #  #
```

```
pranavv@pranavv:~$
```

6) tty

```
pranavv@pranavv:~$ tty
/dev/pts/0
pranavv@pranavv:~$
```

7) script

```

pranavv@pranavv:~$ script
Script started, file is typescript
pranavv@pranavv:~$ echo hello
hello
pranavv@pranavv:~$ exit 2
exit
Script done, file is typescript
pranavv@pranavv:~$ cat typescript
Script started on 2021-01-27 22:23:40+05:30 [TERM="xterm-256color" TTY="/dev/pts/0" COLUMNS="152" LINES="45"]
pranavv@pranavv:~$ echo hello
hello
pranavv@pranavv:~$ exit 2
exit
Script done on 2021-01-27 22:23:59+05:30 [COMMAND_EXIT_CODE="2"]
pranavv@pranavv:~$

```

8) man



```

PwD(1) User Commands PwD(1)
NAME
    pwd - print name of current/working directory
SYNOPSIS
    pwd [OPTION]...
DESCRIPTION
    Print the full filename of the current working directory.
    -L, --logical
        use PwD from environment, even if it contains symlinks
    -P, --physical
        avoid all symlinks
    --help
        display this help and exit
    --version
        output version information and exit
    If no option is specified, -P is assumed.
    NOTE: your shell may have its own version of pwd, which usually supersedes the version described here. Please refer to your shell's documentation for details about the options it supports.
AUTHOR
    Written by Jiri Meisinger.
REPORTING BUGS
    GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
    Report translation bugs to <https://translationproject.org/team/>
COPYRIGHT
    Copyright © 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <https://www.gnu.org/licenses/gpl.html>.
    This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law.
SEE ALSO
    getcwd(3)
    Full documentation at: <https://www.gnu.org/software/coreutils/pwd>
    or available locally via: info '(coreutils) pwd invocation'
GNU coreutils 8.30
-
Manual page pwd(1) line 1/44 (END) (press h for help or q to quit)

```

2) Write down different options for cal commands. (use \$man cal)

Ans.

-h : Turns of highlighting of today.

-J : Display Julian Calendar, if combined with the -o option, display date of Orthodox Easter according to the Julian Calendar.

-e : Display date of Easter (for western churches).

-j : Display Julian days (days one-based, numbered from January 1).

-m month : Display the specified month. If month is specified as a decimal number, appending 'f' or 'p' displays the same month of the following or previous year respectively.

-o : Display date of Orthodox Easter (Greek and Russian Orthodox Churches).

-p : Print the country codes and switching days from Julian to Gregorian Calendar as they are assumed by ncal. The country code as determined from the local environment is marked with an asterisk.

-s country_code : Assume the switch from Julian to Gregorian Calendar at the date associated with the country_code. If not specified, ncal tries to guess the switch date from the local environment or falls back to September 2, 1752. This was when Great Britain and her colonies switched to the Gregorian Calendar.

-w : Print the number of the week below each week column.

-y : Display a calendar for the specified year. This option is implied when a year but no month are specified on the command line.

-3 : Display the previous, current and next month surrounding today.

-1 : Display only the current month. This is the default.

-A number : Months to add after. The specified number of months is added to the end of the display. This is in addition to any date range selected by the -y, -3, or -1 options. For example, "cal -y -B2 -A2" shows everything from November of the previous year to February of the following year. Negative numbers are allowed, in which case the specified number of months is sub-tracted. For example, "cal -y -B-6" shows July to December. And "cal -A11" simply shows the next 12 months.

-B : Months to add before. The specified number of months is added to the beginning of the display. See -A for examples.

-C : Completely switch to cal mode. For cal like output only, use -b instead.

-N : Switch to ncal mode.

-d yyyy-mm : Use yyyy-mm as the current date (for debugging of date selection).

-H yyyy-mm-dd : Use yyyy-mm-dd as the current date (for debugging of highlighting).

-M : Weeks start on Monday.

-S : Weeks start on Sunday.

-b : Use oldstyle format for ncal output.

3) Write options of date command.(Use \$man date).

Ans.

-d, --date=STRING : display time described by STRING, not 'now'

--debug : annotate the parsed date, and warn about questionable usage to stderr

-f, --file=DATEFILE : like --date; once for each line of DATEFILE

-I[FMT], --iso-8601[=FMT] : output date/time in ISO 8601 format. FMT='date' for date only (the default), 'hours', 'minutes', 'seconds', or 'ns' for date and time to the indicated precision. Example: 2006-08-14T02:34:56-06:00

-R, --rfc-email : output date and time in RFC 5322 format. Example: Mon, 14 Aug 2006 02:34:56 -0600

--rfc-3339=FMT : output date/time in RFC 3339 format. FMT='date', 'seconds', or 'ns' for date and time to the indicated precision. Example: 2006-08-14 02:34:5606:00

-r, --reference=FILE : display the last modification time of FILE

-s, --set=STRING : set time described by STRING

-u, --utc, --universal : print or set Coordinated Universal Time (UTC)

--help : display this help and exit

--version : output version information and exit

PRACTICAL RELATED QUESTIONS :

1) How you record all the activities performed by the user.

Ans. Script command is used to record all the terminal activities. By default all the information is saved in file typescript. And to stop the typescript exit command is used.

Syntax: script [options] [file]

2) Give a command to display calendar for month of January.

Ans.

- While the cal/ncal commands display the current month by default, you can use the -m command-line option in order to have a specific month displayed.
- This option requires a numeric value (1-12) that represents the month you want the command to display.
- To display the month of January of current year: `cal -m1`
- To display calendar of specific month and year: `cal [ [month] year]`.

3) Give a single command to display the calendar of pervious, current and next month.

Ans. `cal -3` command is used to display the calendar of pervious, current and next month.

4) Give the command to display full week day (eg. Sunday) using date command.

Ans. `%A` is used as Locale's abbreviated full weekday name (e.g., Sunday)

EXERCISE :

1) What is the output of the following commands?

Ans

a) `$cal 04 2019`

```
pranavv@pranavv:~$ cal 04 2019
April 2019
Su Mo Tu We Th Fr Sa
   1  2  3  4  5  6
  7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
pranavv@pranavv:~$
```

b) `$date "+Today's information: %D and %B";cal`

```
pranavv@pranavv:~$ date "+Today's information : %D and %B";cal
Today's information : 01/27/21 and January
January 2021
Su Mo Tu We Th Fr Sa
   1  2
  3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
pranavv@pranavv:~$
```

- c) \$date "+My clock is showing %H hours, %M minutes and %S seconds"

```
pranavv@pranavv:~$ date "+My clock is showing %H hours,%M minutes and %s seconds"
My clock is showing 22 hours,29 minutes and 1611766762 seconds
pranavv@pranavv:~$
```

- d) \$cal -3

```
pranavv@pranavv:~$ cal -3
      December 2020      January 2021      February 2021
Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa  Su Mo Tu We Th Fr Sa
      1  2  3  4  5          1  2          1  2  3  4  5  6
 6  7  8  9 10 11 12    3  4  5  6  7  8  9    7  8  9 10 11 12 13
13 14 15 16 17 18 19    10 11 12 13 14 15 16   14 15 16 17 18 19 20
20 21 22 23 24 25 26    17 18 19 20 21 22 23   21 22 23 24 25 26 27
27 28 29 30 31          24 25 26 27 28 29 30   28
                          31
pranavv@pranavv:~$
```

- e) \$cal -5

```
pranavv@pranavv:~$ cal -5
cal: invalid option -- '5'
Usage: cal [general options] [-jy] [[month] year]
       cal [general options] [-j] [-m month] [year]
       ncal -C [general options] [-jy] [[month] year]
       ncal -C [general options] [-j] [-m month] [year]
       ncal [general options] [-bhJjpywSM] [-H yyyy-mm-dd] [-s country_code] [[month] year]
       ncal [general options] [-bhJeoSM] [year]
General options: [-31] [-A months] [-B months] [-d yyyy-mm]
pranavv@pranavv:~$
```

- f) \$cal -2000

```
pranavv@pranavv:~$ cal -2000
cal: invalid option -- '2'
Usage: cal [general options] [-jy] [[month] year]
       cal [general options] [-j] [-m month] [year]
       ncal -C [general options] [-jy] [[month] year]
       ncal -C [general options] [-j] [-m month] [year]
       ncal [general options] [-bhJjpywSM] [-H yyyy-mm-dd] [-s country_code] [[month] year]
       ncal [general options] [-bhJeoSM] [year]
General options: [-31] [-A months] [-B months] [-d yyyy-mm]
pranavv@pranavv:~$
```

- 2) Give the syntax of commands for displaying the output. (use date)

Ans.

- a) This is Month of the year (Abbreviation for the month and 4-digit year)

Ans. date "+This is %b month of the year %Y"

```
pranavv@pranavv:~$ date "+This is %b month of the year %Y"
This is Jan month of the year 2021
pranavv@pranavv:~$
```

b) This is the th day of this year.

Ans. date "+This is %j th day of this year"

```
pranavv@pranavv:~$ date "+This is %jth day of the year"  
This is 027th day of the year  
pranavv@pranavv:~$
```

CONCLUSION : We successfully implemented commands to perform operations using commands like date, time, cal, clear, banner , tty, script, man.



Practical 3 : Work with multiple Linux terminals and basic commands

PRACTICAL SIGNIFICANCE :

Work with terminal environment to know about users and set their security. It includes current status of all the users like details about all the users who accesses the terminal for particular duration and path of directory.

MINIMUM THEORETICAL BACKGROUND :

Sr. No.	Commands	Syntax	Description
1	who	\$who	It is used to display who are the users connected to our computer currently
2	who am i	\$who am i	Display the details of the current working directory
3	login	login: \$username	Prompt, enter username
4	passwd	\$passwd uname	Sets password for users
5	su(sudo)	\$su ls	Provides super user privileges
6	pwd(Present Working Dir)	\$pwd	To print the complete path of the current working directory

PROGRAM CODE :

List down with all options for who commands and write its description. Note: Write terminal's output in Result Section.

Options for 'who' command	
-a, --all	Same as -b -d --login -p -r -t -T -u Displays all info.
-b, --boot	Time of last system boot
-d, --dead	Print dead processes
-H, --heading	Print line of column headings
--ips	Print ips instead of hostnames. with --lookup, canonicalizes based on stored IP, if available, rather than stored hostname
-l, --login	Print system login processes
--lookup	Attempt to canonicalize hostnames via DNS
-m	Only hostname and user associated with stdin
-p, --process	Print active processes spawned by init
-q, --count	All login names and number of users logged on
-r, --runlevel	Print current runlevel
-s, --short	Print only name, line, and time (default)
-t, --time	Print last system clock change
-T, -w, --mesg	Add user's message status as '+', '-' or '?'
-u, --users	List users logged in

--message	Same as -T
--writable	Same as -T
--help	Display this help and exit
--version	Output version information and exit

RESULT :

```
pranavv@pranavv:~$ who
pranavv  :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who -a
           system boot    2021-01-27 15:24
           run-level 5     2021-01-27 15:25
pranavv  ? :0                2021-01-27 15:25  ?           1308 (:0)
pranavv@pranavv:~$ who -H
NAME     LINE      TIME                COMMENT
pranavv  :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who -l -H
NAME     LINE      TIME                IDLE          PID COMMENT
pranavv@pranavv:~$ who -t -H
NAME     LINE      TIME                PID COMMENT
pranavv@pranavv:~$ who -m -H
NAME     LINE      TIME                COMMENT
pranavv@pranavv:~$ who -q
pranavv
# users=1
pranavv@pranavv:~$ who -boot
who: invalid option -- 'o'
Try 'who --help' for more information.
pranavv@pranavv:~$ who --boot
           system boot    2021-01-27 15:24
pranavv@pranavv:~$ who --dead
pranavv@pranavv:~$ who --ips
pranavv  :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who --login
pranavv@pranavv:~$ who --lookup
pranavv  :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who --process
pranavv@pranavv:~$ who --runlevel
           run-level 5     2021-01-27 15:25
pranavv@pranavv:~$ who --short
pranavv  :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who --time
pranavv@pranavv:~$ who --mesg
pranavv  ? :0                2021-01-27 15:25 (:0)
pranavv@pranavv:~$ who --user
pranavv  :0                2021-01-27 15:25  ?           1308 (:0)
pranavv@pranavv:~$
```

```

pranavv@pranavv:~$ who --help
Usage: who [OPTION]... [ FILE | ARG1 ARG2 ]
Print information about users who are currently logged in.

-a, --all           same as -b -d --login -p -r -t -T -u
-b, --boot          time of last system boot
-d, --dead          print dead processes
-H, --heading       print line of column headings
    --ips           print ips instead of hostnames. with --lookup,
                   canonicalizes based on stored IP, if available,
                   rather than stored hostname
-l, --login         print system login processes
    --lookup        attempt to canonicalize hostnames via DNS
-m                 only hostname and user associated with stdin
-p, --process       print active processes spawned by init
-q, --count         all login names and number of users logged on
-r, --runlevel      print current runlevel
-s, --short         print only name, line, and time (default)
-t, --time          print last system clock change
-T, -w, --mesg      add user's message status as +, - or ?
-u, --users         list users logged in
    --message       same as -T
    --writable       same as -T
    --help          display this help and exit
    --version       output version information and exit

If FILE is not specified, use /var/run/utmp. /var/log/wtmp as FILE is common.
If ARG1 ARG2 given, -m presumed: 'am i' or 'mom likes' are usual.

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Full documentation at: <https://www.gnu.org/software/coreutils/who>
or available locally via: info '(coreutils) who invocation'
pranavv@pranavv:~$

```

PRACTICAL RELATED QUESTIONS :

1) Give command for present working directory.

Ans. pwd

2) State currently login users by command.

Ans. who

3) Acquire the status of super user.

Ans.

```
pranavv@pranavv:~$ sudo -i
[sudo] password for pranavv:
root@pranavv:~# whoami
root
root@pranavv:~# logout
pranavv@pranavv:~$ whoami
pranavv
pranavv@pranavv:~$
```

EXERCISE :

- 1) Acquire the status of super user.

Ans.

```
pranavv@pranavv:~$ sudo -i
[sudo] password for pranavv:
root@pranavv:~# whoami
root
root@pranavv:~# logout
pranavv@pranavv:~$ whoami
pranavv
pranavv@pranavv:~$
```

- 2) Write output for following commands

- i) \$who;clear;whoami
- ii) \$who;tty;date

Ans.

i)

```
pranavv@pranavv:~$ who;clear;whoami
```

```
pranavv
pranavv@pranavv:~$
```



```
pranavv
pranavv@pranavv:~$ who;tty;date
pranavv  :0          2021-01-26 19:08 (:0)
/dev/pts/0
Tuesday 26 January 2021 07:38:05 PM IST
pranavv@pranavv:~$
```

ii)

CONCLUSION : We successfully implemented basic commands with multiple linux terminals.



Practical 4: Working With a) operating system services b) Processes

PRACTICAL SIGNIFICANCE :

The purpose of computer systems is to allow the user to execute programs. So the Operating system provides an environment where the user can conveniently run programs. The user does not have to worry about the memory allocation or multitasking or anything. These things are taken care of by the operating systems.

MINIMUM THEORETICAL BACKGROUND :

An Operating System supplies different kinds of services to both the users and to the programs as well. It also provides application programs (that run within an Operating system) an environment to execute it freely. It provides users the services to run various programs in a convenient manner. File editing is done with the help of any editor. The default editor that comes with the UNIX operating system is called vi (visual editor). The UNIX vi editor is a full screen editor and has two modes of operation: Command mode commands which cause action to be taken on the file, and Insert mode in which entered text is inserted into the file.

Editor : There are many ways to edit files in Unix. Vi Editor works in Unix. This editor enables you to edit lines in context with other lines in the file. You can also use this editor to just read a text file. An improved version of the vi editor which is called the VIM has also been made available now. Here, VIM stands for Vi Improved.

GUI : Linux system provides both the interface that is GUI and CLI. As per the requirement you can decide when you should use the Linux command line interface (CLI) and when you should use a graphical user interface (GUI).

File Handling.

Following are the Processes commands:

Command to add and remove services in your system start-up: use chkconfig

Enter the command to show currently running services.

```
ls /etc/init.d
```

Enter the restart command.

```
sudo systemctl
```

To stop service

```
sudo systemctl,
```

To start service.

```
sudo systemctl
```

PROGRAM CODE :

Check all permissions started on your system. Stop the services which are not required for long time.

We execute following commands on ubuntu operating system:

- 1) Sudo systemctl start acpit
- 2) Sudo systemctl stop ufw
- 3) Sudo systemctl start ufw
- 4) Sudo systemctl restart ufw
- 5) Pwd
- 6) Cd

RESULT :

- cd

```
pranavv@pranavv:~$ cd pnv56
pranavv@pranavv:~/pnv56$ pwd
/home/pranavv/pnv56
pranavv@pranavv:~/pnv56$ cd
pranavv@pranavv:~$
```

- pwd

```
pranavv@pranavv:~$ cd pnv56
pranavv@pranavv:~/pnv56$ pwd
/home/pranavv/pnv56
pranavv@pranavv:~/pnv56$ ls
abc.sh
pranavv@pranavv:~/pnv56$
```

- Sudo systemctl stop ufw

```
pranavv@pranavv:~$ sudo systemctl stop ufw
[sudo] password for pranavv:
pranavv@pranavv:~$ service --statu-all
--statu-all: unrecognized service
pranavv@pranavv:~$ service --status-all
[ + ] acpid
[ - ] alsa-utils
[ - ] anacron
[ + ] apparmor
[ + ] appport
[ + ] avahi-daemon
[ - ] bluetooth
[ - ] console-setup.sh
[ + ] cron
[ + ] cups
[ + ] cups-browsed
[ + ] dbus
[ + ] gdm3
[ + ] grub-common
[ - ] hwclock.sh
[ + ] irqbalance
[ + ] kerneloops
[ - ] keyboard-setup.sh
[ + ] kmod
[ + ] network-manager
[ + ] openvpn
[ - ] plymouth
[ - ] plymouth-log
[ - ] pppd-dns
[ + ] procs
[ - ] pulseaudio-enable-autospawn
[ - ] rsync
[ + ] rsyslog
[ - ] saned
[ - ] speech-dispatcher
[ - ] spice-vdagent
[ + ] udev
[ - ] ufw
[ + ] unattended-upgrades
[ - ] uuuid
[ + ] whoopsie
[ - ] x11-common
pranavv@pranavv:~$
```

- Sudo systemctl start ufw

```
pranavv@pranavv:~$ sudo systemctl start ufw
pranavv@pranavv:~$ service --status-all
[ + ] acpid
[ - ] alsa-utils
[ - ] anacron
[ + ] apparmor
[ + ] apport
[ + ] avahi-daemon
[ - ] bluetooth
[ - ] console-setup.sh
[ + ] cron
[ + ] cups
[ + ] cups-browsed
[ + ] dbus
[ + ] gdm3
[ + ] grub-common
[ - ] hwclock.sh
[ + ] irqbalance
[ + ] kerneloops
[ - ] keyboard-setup.sh
[ + ] kmod
[ + ] network-manager
[ + ] openvpn
[ - ] plymouth
[ - ] plymouth-log
[ - ] pppd-dns
[ + ] procs
[ - ] pulseaudio-enable-autospawn
[ - ] rsync
[ + ] rsyslog
[ - ] saned
[ - ] speech-dispatcher
[ - ] spice-vdagent
[ + ] udev
[ + ] ufw
[ + ] unattended-upgrades
[ - ] uidd
[ + ] whoopsie
[ - ] x11-common
pranavv@pranavv:~$
```

- Sudo systemctl restart ufw

```
pranavv@pranavv:~$ sudo systemctl restart ufw
pranavv@pranavv:~$ service --status-all
[ + ] acpid
[ - ] alsa-utils
[ - ] anacron
[ + ] apparmor
[ + ] apport
[ + ] avahi-daemon
[ - ] bluetooth
[ - ] console-setup.sh
[ + ] cron
[ + ] cups
[ + ] cups-browsed
[ + ] dbus
[ + ] gdm3
[ + ] grub-common
[ - ] hwclock.sh
[ + ] irqbalance
[ + ] kerneloops
[ - ] keyboard-setup.sh
[ + ] kmod
[ + ] network-manager
[ + ] openvpn
[ - ] plymouth
[ - ] plymouth-log
[ - ] pppd-dns
[ + ] procps
[ - ] pulseaudio-enable-autospawn
[ - ] rsync
[ + ] rsyslog
[ - ] saned
[ - ] speech-dispatcher
[ - ] spice-vdagent
[ + ] udev
[ + ] ufw
[ + ] unattended-upgrades
[ - ] uuidd
[ + ] whoopsie
[ - ] x11-common
pranavv@pranavv:~$
```

PRACTICAL RELATED QUESTIONS :

1) List various menus you observed on your system.

Ans. Menu may be defined as the commands and operations in a list which is presented to the user in a computer system or communication system in terms of computing and telecommunication.

The various menus observed in the system are :

- Pop up menu
- Menu bar

- Command Line Interface menu
- Graphical User Interface menu

There are more menus but these are the most required and observed menu on the system.

2) Study the GUI of your Linux system

Ans. Default installation of Ubuntu contains a wide range of software that includes LibreOffice, Firefox, Thunderbird, Transmission, and several lightweight games such as Sudoku and chess. Many additional software packages are accessible from the built in Ubuntu Software (previously Ubuntu Software Center) as well as any other APTbased package management tools. Many additional software packages that are no longer installed by default, such as Evolution, GIMP, Pidgin, and Synaptic, are still accessible in the repositories and installable by the main tool or by any other APT-based package management tool. Cross-distribution snap packages and flatpaks are also available, that both allow installing software, such as some of Microsoft's software, in most of the major Linux operating systems (such as any currently supported Ubuntu version and in Fedora). The default file manager is GNOME Files, formerly called Nautilus. All of the application software installed by default is free software. In addition, Ubuntu redistributes some hardware drivers that are available only in binary format, but such packages are clearly marked in the restricted component.

3) What is difference between GUI and CLI?

Ans.

BASIS	COMMAND LINE INTERFACE(CLI)	GRAPHIC USER INTERFACE(GUI)
Definition	Interaction is by typing commands	Interaction with devices is by graphics and visual components and icons
Understanding	Commands need to be memorized	Visual indicators and icons are easy to understand
Memory	Less memory is required for storage	More memory is required as visual components are involved.
Working Speed	Use of keyboard for commands makes CLI quicker.	Use of mouse for interaction makes it slow
Resources used	Only keyboard	Mouse and keyboard both can be used
Accuracy	High	Comparatively low
Flexibility	Command line interface does not change, remains same over time	Structure and design can change with updates

EXERCISE :

1) What are system calls provided by file management?

Ans. **File Management**

These system calls are responsible for file manipulation such as creating a file, reading a file, writing into a file etc.

For Windows following are the File management system calls

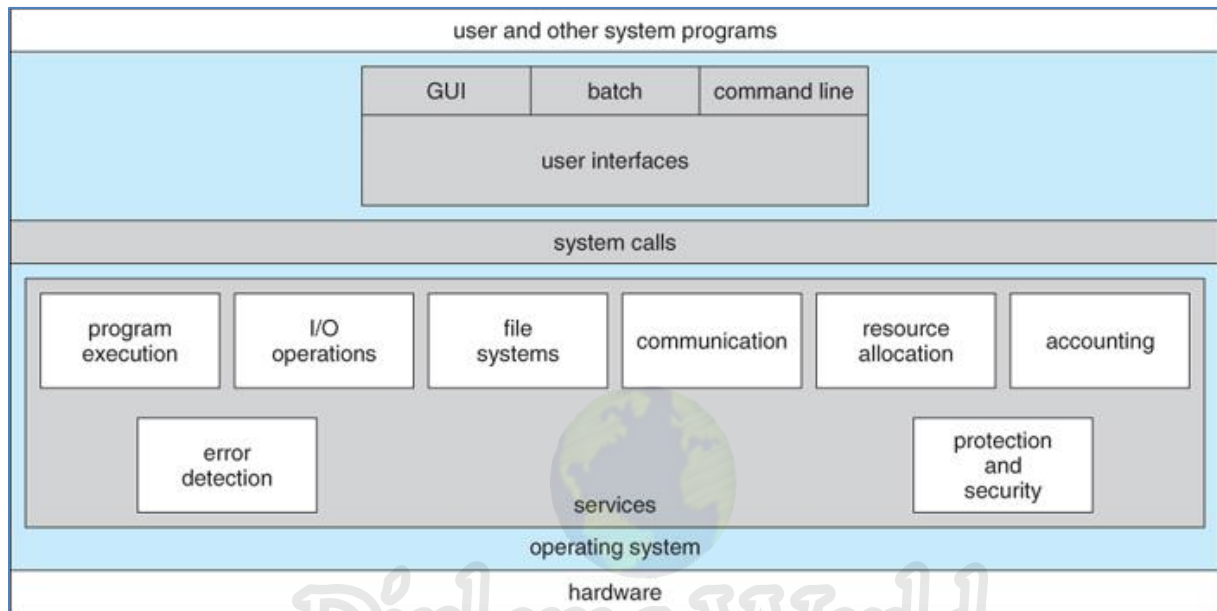
- createFile()
- readFile()
- writeFile()
- closeHandle()

For Linux following are the File management system calls

- open()
- read()
- write()
- close()

2) Draw and explain services provided by operating system

Ans.



Following are a few common services provided by an operating system –

- 1) Program execution
- 2) I/O operations
- 3) File System manipulation
- 4) Communication
- 5) Error Detection
- 6) Resource Allocation
- 7) Protection

1. Program execution : Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process. A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use).
2. I/O Operation : An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the user.
3. File system manipulation : A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives

like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

4. Error handling : Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware.
5. Resource Management : In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job.
6. Protection : Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

3) What are system components of operating system?

Ans.

- File Management.
- Process Management.
- I/O Device Management.
- Network Management.
- Main Memory management.
- Secondary-Storage Management.
- Security Management.

CONCLUSION : We have successfully execute all the commands of Linux operating system of GUI, editor and file handling commands. Commands are as follows :

- Sudo systemctl start acpit
- Sudo systemctl stop ufw
- Sudo systemctl start ufw
- Sudo systemctl restart ufw
- Pwd
- Cd



Practical 5 : Execute process commands - ps, wait, sleep, exit, kill

PRACTICAL SIGNIFICANCE :

Process is program in execution. ps commands is used to manage and to see the active processes on Unix operating system. Ps basically means 'Process Status' which is used to display currently running processes in Unix operating system.

MINIMUM THEORETICAL BACKGROUND :

1) ps

- Program running on the system are called as processes. For examining these processes, users need to know about ps command.
- This command is useful in producing the information for all the processes that are running on the machine.

- The ps command is not easy to use as it presents number of parameters that makes it complicated.
- 2) **wait**
- The wait is one of the built-in commands of Linux. It waits for the completion of any running process.
 - Waits until all background processes are completed and then exits.
- 3) **sleep**
- Sleep command is used to introduce delay for a specific time. This command helps to pause a process for a given time.
 - Suffixes can be used with command to specify exact time. “s” can be used for seconds. “m” can be used for minutes, “h” can be used for hours and “d” can be used for days.
- 4) **exit**
- Generally shell script will exit with the last command’s exit status.
 - Used to quite the shell.
- 5) **kill**
- Kill command is used to kill the process. User has to specify process ID for killing the process.
 - Kill command sends TERMINATE signal to all the process IDs that are listed on command line.
 - It may be difficult to use kill command as it uses process ID instead of its command name.
 - For killing the process user should be the owner of process or must have logged in from root user.

PROGRAM CODE :

- **\$ ps**

```
pranavv@pranavv:~/pnv56$ ps
  PID TTY          TIME CMD
 2050 pts/0        00:00:00 bash
 3890 pts/0        00:00:00 ps
```

- **\$ sleep 10**

```
pranavv@pranavv:~$ sleep 10
pranavv@pranavv:~$
```

- **\$ kill**

```
pranavv@pranavv:~/pnv56$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH    29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

➤ **\$ wait**

```
utkarsha@utkarsha-VirtualBox:~$ cd ~/utk54
utkarsha@utkarsha-VirtualBox:~/utk54$ bash wait.sh
testing wait command1
testing wait command2
Job 1 exited with status 0
Job 2 exited with status 0
utkarsha@utkarsha-VirtualBox:~/utk54$
```

➤ **\$ exit**

```
pranavv@pranavv:~$ exit
```

PRACTICAL RELATED QUESTIONS :

1) What is name of your login shell?

Ans. ps -p \$\$ – Display your current shell name reliably.

```
pranavv@pranavv:~/pnv56$ ps -p$$
  PID TTY          TIME CMD
 2050 pts/0    00:00:00 bash
```

2) What are various options of kill command?

Ans.

1. \$kill -l : To display all the available signals

```
pranavv@pranavv:~/pnv56$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

2. \$kill pid : To show how to use a PID with the kill command.

Syntax : \$kill pid

```
pranavv@pranavv:~$ ps
  PID TTY          TIME CMD
 3062 pts/0    00:00:00 bash
 3072 pts/0    00:00:00 ps
pranavv@pranavv:~$ kill 3072
bash: kill: (3072) - No such process
```

3. kill -s : To show how to send signal to processes.
Syntax : kill {-signal | -s signal} pid
4. kill -L : This command is used to list available signals in a table format.
Syntax : kill {-l | --list[=signal] | -L | --table}

3) What are various options of ps command?

Ans.

Option	Description
-a	Displays all processes on a terminal, with the exception of group .
-c	Displays scheduler data.
-d	Displays all processes with the exception of session leaders.
-e	Displays all processes.
-f	Displays a full listing.
-glist	Displays data for the list of group leader IDs.
-j	Displays the process group ID and session ID.

- l Displays a long listing
- plist Displays data for the list of process IDs.
- slist Displays data for the list of session leader IDs.
- tlist Displays data for the list of terminals.
- ulist Displays data for the list of usernames.

4) Explain about exit command.

Ans. exit command in linux is used to exit the shell where it is currently running. It takes one more parameter as [N] and exits the shell with a return of status N. If n is not provided, then it simply returns the status of last command that is executed.

Syntax : exit [n]

```
pranavv@pranavv:~$ exit
```

5) List the system calls for process management

Ans. System calls Description

fork() : To create a new process

exec() : To execute a new program in a process

wait() : To wait until a created process completes its execution

exit() : To exit from a process execution

getpid() : To get a process identifier of the current process

getppid() : To get parent process identifier

nice() : To bias the existing priority of a process

brk() : To increase/decrease the data segment size of a process

EXERCISE :

- 1) Observe the output of following commands
\$sleep 30; date

\$echo \$\$

Ans.

```
pranavv@pranavv:~/pnv56$ sleep 30;date
Tuesday 26 January 2021 08:03:54 PM IST

pranavv@pranavv:~/pnv56$ echo $$
2050
```

2) Display full listing of all processes running on your terminal.

Ans.

```
pranavv@pranavv:~/pnv56$ ps
  PID TTY          TIME CMD
 2050 pts/0    00:00:00 bash
 3890 pts/0    00:00:00 ps
```

3) Write output of following command.
\$sleep 60; banner GOOD

Ans.

```
pranavv@pranavv:~/pnv56$ sleep 60; banner GOOD
#####
# # # # #
# # # # #
# # # # #
# # # # #
# # # # #
#####

pranavv@pranavv:~/pnv56$
```

4) Difference between Wait and Sleep

Ans.

Key	Wait	Sleep
Class	Wait() method belongs to Object class	Sleep() method belongs to Thread class
Lock Release	Wait() releases the lock on an object	It does not release lock on an object
Calling context	Wait() can be called on object itself	Sleep() can be called on Thread.
Wake-up condition	Until call notify(), notifyAll()	Until atleast time expire or

	from object	call interrupt.
spurious wakeups	Program can get spurious wakeups	It will not get spurious wakeups.

5) Write all the process commands

Ans.

1. ps - If you run ps command without any arguments, it displays processes for the current shell.
2. Wait -When wait command is executed with \$process_id then the next command will wait for completing the task of the first echo command. The second wait command is used with '\$!' ' and this indicate the process id of the last running process.
3. Sleep - command is used to delay for a fixed amount of time during the execution of any script.
4. Kill -kill command in Linux (located in /bin/kill), is a built-in command which is used to terminate processes manually.
5. Exit- used to quite the shell.

CONCLUSION : In this practical we successfully install virtual box, Ubuntu on our laptop. And successfully Run the basic process commands on Ubuntu.

Practical 6 : Execute file and directory manipulation commands

PRACTICAL SIGNIFICANCE :

Basically the operations performed in unix or linux are done on a prompt or terminal which is not stored anywhere. but most of the commands are return to store the information. One way is to store the information in file. Various operation that can be performed on a file are create , open, read ,write ,move,close and rename.

MINIMUM THEORETICAL BACKGROUND :

File and directory manipulation commands :

- **ls Command:** It is used to lists files in current directory.

Syntax: \$ls

Options	Meanings
----------------	-----------------

ls -a	list all files including hidden files starts with ‘.’
-------	---

ls -d	list directories – with ‘*’
-------	-----------------------------

ls -i list files in I node index number
ls -l list with long format – show permissions

Example:

\$ls -l abc.txt list contains of abc.txt files
\$ls -l list of all files along with permission given to it
\$ls ??? list the names of the files with exact three character in it name
\$ls d*s list of file names starts with d and ends with s
\$ls -l gives full listing of all files and directories

• **Rm commands:** it is used to remove files

Syntax: \$rm filename

Example:

\$rm abc.txt removes abc.txt
\$rm abc.txt xyz.txt removes abc.txt and xyz.txt files
\$rm * removes all files

• **Mv commands:** it is used to move a file from one location to another

Syntax: Mv [option] source destination

(consider 3 files having name abc.txt xyz.txt and pqr.txt)

to rename the file abc.txt to aaa.txt(not exists)

```
$ls
Abc.txt xyz.txt pqr.txt

$mv abc.txt aaa.txt

$ls
Xyz.txt pqr.txt aaa.txt
```

Options:

1. **-i(interactive)** : the mv command with -I option asks for confirmation \$mv -I abc.txt aab.txt

• **Cp command:** it is used to copy a file to destination file. If the command contains two file names, then it copies the contents of first file to the second file. If the 2nd file doesn't exist then first it creates one and content is copied to it. But if it existed then it is simply overwritten without any warning.

Syntax : \$cp source file destination file

Example:

```
$ls
```

```
xyz.txt
```

```
$cp xyz.txt zzz.txt
```

```
Check it now
```

```
$ls xyz.txt zzz.txt
```



Diploma World

• **Join command:** It is used for joining lines of two files on the basis of common field in each file called as a key.

Syntax: \$join [-t -o]filename1 filename2

-t field separator

-o Specify the column each table that must be included

Example :

```
$cat stud1
```

```
Ajay 1990
```

```
Vijay 1992
```

```
Sujit 1991
```

```
$cat stud2
```

Ajay 50

Vijay 62

Sujit 70

\$join -t"." stud1 stud2

Ajay 1990:50

Vijay 1992:62

Sujit 1991:70

- **Split command:** It is used to split the large file into smaller files. Default size is 1000 lines/file.

Syntax : \$split -n filename tag name

-n no.of lines in each smaller file.

-tag name by default the split builds the o/p files named xaa , xab , xac

It tags name specified , it replaces x with that tag name

Example :

Split -100 student

Check it now

-ls

Xaa xab xac

- **Cat command :** It is used to create the file and displaying the contents of file/files.

Syntax: \$cat > sample(to create a file)

This is a simple file in UNIX

<Cntrl d>

\$cat sample

Where Cat is a command and sample is a argument. this Is sample file in UNIX. User can display contents of more than one file and called as concatenation.

\$cat list1.txt and list2.txt (list2 files at a time)

• **Head command:** it is used to read the first 10 lines of file.(10 lines by default) the no.of lines to be displayed maybe specified in the head command.

Syntax: \$head -n filename (-n means how many lines to be displayed)

Example :

\$head -20 student

Displays the 1st 20 lines of file student.

• **Tail command:** it is used to print last few numbers of lines(10 by default) of a certain file, then terminates.

Syntax: \$tail{[+/-]n[lbc]}filename

Options

meaning

+n Displays all line starting from nth line

-n displays n line from end of the file

L indicates line

B indicates blocks

C indicate character

Example :

\$tail -20l student

Displays last 20 lines of student file.

\$tail +20l student

Displays all lines starting from 20th lines of the student file.

- **Touch command:** it is the easiest way to create new,empty files

Syntax: \$touch filename

Example :

```
$touch aaa
```

File saving and redirection operator : Generally UNIX commands are entered at the keyboard and output resulting from these commands are displayed on the computer screen. The standard input and standard output are actually taken or saved to files.

Example :

Output of 1st command is taken as input for 2nd command. In this example output of ls command is saved into a file out.txt

```
$ls > out.txt
```

Combine 2 text files to another file :

```
$cat list1.txt list2.txt > todo.txt
```

In this example contents of file list1.txt and list2.txt is saved to todo.txt file.

```
$cat list1.txt >> list2.txt
```

It appends the contents list1.txt to list2.txt

It adds the contents at the end of file.

Now check it

```
$ls list2.txt
```

PROGRAM CODE :

- 1) Create three files a1,a2,a3.

Ans.

```
pranavv@pranavv:~/p6$ >a.text
pranavv@pranavv:~/p6$ >b.text
pranavv@pranavv:~/p6$ >c.text
pranavv@pranavv:~/p6$ ls
a.text b.text c.text p61,p62,p63
pranavv@pranavv:~/p6$
```

2) Apply different commands like :

- | | | |
|----------|--------|---------|
| 1)ls | 2)mv | 3)cp |
| 4)rm | 5)join | 6)split |
| 7)cat | 8)head | 9)tail |
| 10)touch | | |

and check the list of files at the ends.

Ans.

1) ls

```
pranavv@pranavv:~/p6$ ls
abc.sh  wait.sh
pranavv@pranavv:~/p6$
```

2) mv

```
pranavv@pranavv:~/p6$ mv c.txt b.txt
pranavv@pranavv:~/p6$ mv b.txt
a.txt b.txt p61/
pranavv@pranavv:~/p6$ mv b.txt home/pranavv/p6
mv: cannot move 'b.txt' to 'home/pranavv/p6': No such file or directory
pranavv@pranavv:~/p6$ mv b.txt home/pranavv/p6/p61
mv: cannot move 'b.txt' to 'home/pranavv/p6/p61': No such file or directory
pranavv@pranavv:~/p6$ ls
a.txt b.txt p61
pranavv@pranavv:~/p6$ rm p61
rm: cannot remove 'p61': Is a directory
pranavv@pranavv:~/p6$ rmdir p61
pranavv@pranavv:~/p6$ mkdir p61
pranavv@pranavv:~/p6$ mv b.txt home/pranavv/p6/p61
mv: cannot move 'b.txt' to 'home/pranavv/p6/p61': No such file or directory
pranavv@pranavv:~/p6$ mv b.txt /home/pranavv/p6/p61
pranavv@pranavv:~/p6$ cd p61
pranavv@pranavv:~/p6/p61$ ls
b.txt
pranavv@pranavv:~/p6/p61$
```

3) cp

```
pranavv@pranavv:~/p6/p61$ cd
pranavv@pranavv:~$ cd p6
pranavv@pranavv:~/p6$ cp a.txt p61
pranavv@pranavv:~/p6$ cd p61
pranavv@pranavv:~/p6/p61$ ls
a.txt b.txt
pranavv@pranavv:~/p6/p61$
```

4) rm


```
pranavv@pranavv:~/p6/p61$ ls
a.txt b.txt
pranavv@pranavv:~/p6/p61$ rm a.txt
pranavv@pranavv:~/p6/p61$ ls
b.txt
pranavv@pranavv:~/p6/p61$
```

5) join

```
pranavv@pranavv:~/p6/p61$ ls
a.txt b.txt
pranavv@pranavv:~/p6/p61$ cat a.txt
1 abc
2 pqr
3 xyz
4 pranavibhute

pranavv@pranavv:~/p6/p61$ cat b.txt
1 9112
2 9113
3 9114
4 9115

pranavv@pranavv:~/p6/p61$ join a.txt b.txt
1 abc 9112
2 pqr 9113
3 xyz 9114
4 pranavibhute 9115
pranavv@pranavv:~/p6/p61$
```

6) split

```
pranavv@pranavv:~/p6/p61$ ls
abc.sh ab.txt wait.sh
pranavv@pranavv:~/p6/p61$ split ab.txt
pranavv@pranavv:~/p6/p61$ ls
abc.sh ab.txt wait.sh xaa
pranavv@pranavv:~/p6/p61$
```

7) cat

```
pranavv@pranavv:~/p6/p61$ cat ab.txt
hello everyone
pranavv@pranavv:~/p6/p61$
```

8) head

```
pranavv@pranavv:~/p6/p61$ head -n 2 a.txt
1 abc
2 pqr
```

9) tail

```
pranavv@pranavv:~/p6/p61$ tail -n 2 a.txt
4 pranavibhute
pranavv@pranavv:~/p6/p61$
```

10) touch

```
pranavv@pranavv:~/p6/p61$ ls
a1.txt Abhay Abhay.txt Amey.txt a.txt b.txt xaa
pranavv@pranavv:~/p6/p61$ touch abc.txt
pranavv@pranavv:~/p6/p61$ ls
a1.txt abc.txt Abhay Abhay.txt Amey.txt a.txt b.txt xaa
pranavv@pranavv:~/p6/p61$
```

PRACTICAL RELATED QUESTIONS :

- 1) What are different options of ls commands? Write down the command along with the option and note down the output.(Use \$man ls commands to check options)

Ans.

Answer: ls is a Linux shell command that lists directory contents of files and directories.

Syntax: \$ ls [options] [file|dir]

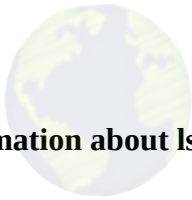
```
pranavv@pranavv:~/p6/p61$ ls
a1.txt a.txt b.txt xaa
pranavv@pranavv:~/p6/p61$ ls--
ls--: command not found
pranavv@pranavv:~/p6/p61$ ls --
a1.txt a.txt b.txt xaa
pranavv@pranavv:~/p6/p61$ ls -a
. .. a1.txt a.txt b.txt xaa
pranavv@pranavv:~/p6/p61$ ls -l
total 12
-rw-rw-r-- 1 pranavv pranavv  0 Jan 26 20:30 a1.txt
-rw-rw-r-- 1 pranavv pranavv 34 Jan 26 20:29 a.txt
-rw-rw-r-- 1 pranavv pranavv 29 Jan 26 20:22 b.txt
-rw-rw-r-- 1 pranavv pranavv 34 Jan 26 20:25 xaa
pranavv@pranavv:~/p6/p61$ ls ../
p61
pranavv@pranavv:~/p6/p61$
```

Examples:

- 1) List directory /home/user/Documents/Books with absolute path.
-\$ ls /home/user/Documents/Books
- 2) List root directory:
-\$ ls /

- 3) List parent directory:
-\$ ls ..
- 4) List user's home directory (e.g: /home/user):
-\$ ls ~
- 5) List with long format:
-\$ ls -l
- 6) Show hidden files:
-\$ ls -a
- 7) List with long format and show hidden files:
-\$ ls -la
- 8) Sort by date/time:
-\$ ls -t
- 9) Sort by file size:
-\$ ls -S
- 10) List all subdirectories:
-\$ ls *

Run \$ man ls OR \$ ls --help to get all information about ls command.



Diploma World

```

pranavv@pranavv:~/p6/p61$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
-a, --all                do not ignore entries starting with .
-A, --almost-all        do not list implied . and ..
    --author             with -l, print the author of each file
-b, --escape             print C-style escapes for nongraphic characters
    --block-size=SIZE    with -l, scale sizes by SIZE when printing them;
                        e.g., '--block-size=M'; see SIZE format below
-B, --ignore-backups     do not list implied entries ending with ~
-c                       with -lt: sort by, and show, ctime (time of last
                        modification of file status information);
                        with -l: show ctime and sort by name;
                        otherwise: sort by ctime, newest first
-C                       list entries by columns
    --color[=WHEN]       colorize the output; WHEN can be 'always' (default
                        if omitted), 'auto', or 'never'; more info below
-d, --directory          list directories themselves, not their contents
-D, --dired              generate output designed for Emacs' dired mode
-f                       do not sort, enable -aU, disable -ls --color
-F, --classify           append indicator (one of */=>@|) to entries
    --file-type           likewise, except do not append '*'
    --format=WORD         across -x, commas -m, horizontal -x, long -l,
                        single-column -1, verbose -l, vertical -C
    --full-time           like -l --time-style=full-iso
-g                       like -l, but do not list owner
    --group-directories-first
                        group directories before files;
                        can be augmented with a --sort option, but any
                        use of --sort=none (-U) disables grouping
-G, --no-group           in a long listing, don't print group names
-h, --human-readable     with -l and -s, print sizes like 1K 234M 2G etc.
    --si                 likewise, but use powers of 1000 not 1024
-H, --dereference-command-line
                        follow symbolic links listed on the command line
    --dereference-command-line-symlink-to-dir
                        follow each command line symbolic link
                        that points to a directory
--hide=PATTERN           do not list implied entries matching shell PATTERN
                        (overridden by -a or -A)
--hyperlink[=WHEN]       hyperlink file names; WHEN can be 'always'
                        (default if omitted), 'auto', or 'never'
--indicator-style=WORD   append indicator with style WORD to entry names:

```

```

-i, --inode          file-type (--file-type), classify (-f)
                     print the index number of each file
-I, --ignore=PATTERN do not list implied entries matching shell PATTERN
-k, --kibibytes      default to 1024-byte blocks for disk usage;
                     used only with -s and per directory totals
-l                  use a long listing format
-L, --dereference    when showing file information for a symbolic
                     link, show information for the file the link
                     references rather than for the link itself
-m                  fill width with a comma separated list of entries
-n, --numeric-uid-gid like -l, but list numeric user and group IDs
-N, --literal        print entry names without quoting
-o                  like -l, but do not list group information
-p, --indicator-style=slash
                     append / indicator to directories
-q, --hide-control-chars print ? instead of nongraphic characters
    --show-control-chars show nongraphic characters as-is (the default,
                           unless program is 'ls' and output is a terminal)
-Q, --quote-name      enclose entry names in double quotes
    --quoting-style=WORD use quoting style WORD for entry names:
                           literal, locale, shell, shell-always,
                           shell-escape, shell-escape-always, c, escape
                           (overrides QUOTING_STYLE environment variable)
-r, --reverse         reverse order while sorting
-R, --recursive       list subdirectories recursively
-s, --size            print the allocated size of each file, in blocks
-S                  sort by file size, largest first
    --sort=WORD       sort by WORD instead of name: none (-U), size (-S),
                           time (-t), version (-v), extension (-X)
    --time=WORD       with -l, show time as WORD instead of default
                           modification time: atime or access or use (-u);
                           ctime or status (-c); also use specified time
                           as sort key if --sort=time (newest first)
    --time-style=TIME_STYLE time/date format with -l; see TIME_STYLE below
-t                  sort by modification time, newest first
-T, --tabsize=COLS    assume tab stops at each COLS instead of 8
-u                  with -lt: sort by, and show, access time;
                     with -l: show access time and sort by name;
                     otherwise: sort by access time, newest first
-U                  do not sort; list entries in directory order
-v                  natural sort of (version) numbers within text
-w, --width=COLS      set output width to COLS. 0 means no limit
-x                  list entries by lines instead of by columns
-X                  sort alphabetically by entry extension
-Z, --context          print any security context of each file
-1                  list one file per line. Avoid '\n' with -q or -b
    --help            display this help and exit
    --version         output version information and exit

```

```

-w, --width=COLS      set output width to COLS. 0 means no limit
-x                  list entries by lines instead of by columns
-X                  sort alphabetically by entry extension
-Z, --context          print any security context of each file
-1                  list one file per line. Avoid '\n' with -q or -b
    --help            display this help and exit
    --version         output version information and exit

```

The SIZE argument is an integer and optional unit (example: 10K is 10*1024). Units are K,M,G,T,P,E,Z,Y (powers of 1024) or KB,MB,... (powers of 1000).

The TIME_STYLE argument can be full-iso, long-iso, iso, locale, or +FORMAT. FORMAT is interpreted like in date(1). If FORMAT is FORMAT1<newline>FORMAT2, then FORMAT1 applies to non-recent files and FORMAT2 to recent files. TIME_STYLE prefixed with 'posix-' takes effect only outside the POSIX locale. Also the TIME_STYLE environment variable sets the default style to use.

Using color to distinguish file types is disabled both by default and with --color=never. With --color=auto, ls emits color codes only when standard output is connected to a terminal. The LS_COLORS environment variable can change the settings. Use the dircolors command to set it.

Exit status:
0 if OK,
1 if minor problems (e.g., cannot access subdirectory),
2 if serious trouble (e.g., cannot access command-line argument).

GNU coreutils online help: <<https://www.gnu.org/software/coreutils/>>
Full documentation at: <<https://www.gnu.org/software/coreutils/ls>>
or available locally via: info '(coreutils) ls invocation'

pranav@pranav:~/p6/p61\$

What are different options of mv commands?

Ans.

```
pranavv@pranavv:~$ mv --help
Usage: mv [OPTION]... [-T] SOURCE DEST
or: mv [OPTION]... SOURCE... DIRECTORY
or: mv [OPTION]... -t DIRECTORY SOURCE...
Rename SOURCE to DEST, or move SOURCE(s) to DIRECTORY.

Mandatory arguments to long options are mandatory for short options too.
  --backup[=CONTROL]  make a backup of each existing destination file
  -b                  like --backup but does not accept an argument
  -f, --force          do not prompt before overwriting
  -i, --interactive    prompt before overwrite
  -n, --no-clobber     do not overwrite an existing file
If you specify more than one of -i, -f, -n, only the final one takes effect.
  --strip-trailing-slashes  remove any trailing slashes from each SOURCE
                           argument
  -S, --suffix=SUFFIX  override the usual backup suffix
  -t, --target-directory=DIRECTORY  move all SOURCE arguments into DIRECTORY
  -T, --no-target-directory  treat DEST as a normal file
  -u, --update          move only when the SOURCE file is newer
                           than the destination file or when the
                           destination file is missing
  -v, --verbose         explain what is being done
  -Z, --context         set SELinux security context of destination
                           file to default type
  --help               display this help and exit
  --version             output version information and exit

The backup suffix is '~', unless set with --suffix or SIMPLE_BACKUP_SUFFIX.
The version control method may be selected via the --backup option or through
the VERSION_CONTROL environment variable. Here are the values:

  none, off           never make backups (even if --backup is given)
  numbered, t         make numbered backups
  existing, nil       numbered if numbered backups exist, simple otherwise
  simple, never       always make simple backups

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Full documentation at: <https://www.gnu.org/software/coreutils/mv>
or available locally via: info '(coreutils) mv invocation'
pranavv@pranavv:~$
```

It has two distinct functions:

- (i) It rename a file or folder.
- (ii) It moves group of files to different directory.

i. To rename the file a.txt to geek.txt(not exist)-

Syntax: mv [Option] source destination

Code:

```
$ ls
```

```
a.txt b.txt c.txt d.txt
```

```
$ mv a.txt geek.txt
```

```
$ ls
```

```
b.txt c.txt d.txt geek.txt
```

ii. moving geeks.txt to b.txt(exist):

Code:

```
$ ls
```

```
b.txt c.txt d.txt geek.txt
```

```
$ cat geek.txt
```

```
India
```

```
$ cat b.txt
```

```
geeksforgeeks
```

```
$ mv geek.txt b.txt
```

```
$ ls
```

```
b.txt c.txt d.txt
```

```
$ cat b.txt
```

```
India
```

2) What is use of split command?

Ans. Split command in Linux is used to split large files into smaller files. It splits the files into 1000 lines per file(by default) and even allows users to change the number of lines as per requirement.

Syntax: split [options] name_of_file prefix_for_new_files

3) How to use join command?

Ans. join command is used to join the two files based on a key field present in both the files.

Syntax: \$join [OPTION] FILE1 FILE2

EXERCISE :

1) Write output of following commands :

- i. Display all files names which starts with 'a' and ends with 'y'.
- ii. Enlist all the files beginning with 'm' and ending with any range of 1 to 5.
- iii. Show the contents of files whose file names contains exactly two characters.
- iv. Create a file ABCD.txt, create a copy with XXX.txt. Rename the original file with AACD.txt. Delete the file XXX.txt.
- v. Display the inodes of any two files at the same time.

Ans.

- i. Display all files names which starts with 'a' and ends with 'y'.

```
pranavv@pranavv:~/p6/p61$ ls A*y
Abhay
pranavv@pranavv:~/p6/p61$
```

- ii. Enlist all the files beginning with 'm' and ending with any range of 1 to 5.

```
pranavv@pranavv:~/p6$ ls
n1 n10 n2 n3 n4 n5 n6 n7 n8 n9 p61
pranavv@pranavv:~/p6$ ls m*[1-5]
n1:
n2:
n3:
n4:
n5:
pranavv@pranavv:~/p6$
```

- iii. Show the contents of files whose file names contains exactly two characters.

```
pranavv@pranavv:~/Documents$ ls|awk 'length($0)==2{print $0}'
CO
ME
pranavv@pranavv:~/Documents$ >ab
pranavv@pranavv:~/Documents$ ls|awk 'length($0)==2{print $0}'
ab
CO
ME
pranavv@pranavv:~/Documents$
```


- iv. Create a file ABCD.txt, create a copy with XXX.txt. Rename the original file with AACD.txt. Delete the file XXX.txt.

```
pranavv@pranavv:~/p6$ echo "this is a sample file" >sample.txt
pranavv@pranavv:~/p6$ ls
m1 m10 m2 m3 m4 m5 m6 m7 m8 m9 p61 sample.txt
pranavv@pranavv:~/p6$ cat sample.txt
this is a sample file
pranavv@pranavv:~/p6$ cp sample.txt ABCD.txt
pranavv@pranavv:~/p6$ ls
ABCD.txt m1 m10 m2 m3 m4 m5 m6 m7 m8 m9 p61 sample.txt
pranavv@pranavv:~/p6$ mv sample.txt demo.txt
pranavv@pranavv:~/p6$ ls
ABCD.txt demo.txt m1 m10 m2 m3 m4 m5 m6 m7 m8 m9 p61
pranavv@pranavv:~/p6$
```

- v. Display the inodes of any two files at the same time.

```
pranavv@pranavv:~/p6$ ls
abc.sh ab.txt wait.sh xaa
pranavv@pranavv:~/p6$ ls -li ab.txt
920281 ab.txt
pranavv@pranavv:~/p6$ ls -li ab.txt abc.sh
920286 abc.sh 920281 ab.txt
pranavv@pranavv:~/p6$
```

- 2) List all file processing commands

Ans.

- a) awk
- b) grep
- c) head
- d) tail
- e) sort
- f) uniq
- g) less
- h) more
- i) tr
- j) pr
- k) fmt
- l) Sed

- 3) How many lines will be displayed with head command if number is not specified.

Ans. 10 lines

- 4) Create two files chapter1 and chapter2 by asking the user before overwrite.
- Copy contents of chapter1 and chapter2 by asking the user before overwrite.
 - Display inodes of two files.
 - Rename the file 'chapter1' to 'Lesson1'

Ans.

- i. Copy contents of chapter1 and chapter2 by asking the user before overwrite.

```
pranavv@pranavv:~/Templates$ cat lesson1
This is lesson 1
pranavv@pranavv:~/Templates$ cat lesson2
This is lesson 2
pranavv@pranavv:~/Templates$ cp lesson1 lesson2
pranavv@pranavv:~/Templates$ cat lesson1
This is lesson 1
pranavv@pranavv:~/Templates$ cat lesson2
This is lesson 1
pranavv@pranavv:~/Templates$
```

- ii. Display inodes of two files.

```
pranavv@pranavv:~/Templates$ ls -li lesson1 lesson2
1054700 lesson1 1055221 lesson2
pranavv@pranavv:~/Templates$
```

- iii. Rename the file 'chapter1' to 'Lesson1'

```
pranavv@pranavv:~/Templates$ ls
lesson1 lesson2
pranavv@pranavv:~/Templates$ mv lesson1 chapter1
pranavv@pranavv:~/Templates$ ls
chapter1 lesson2
pranavv@pranavv:~/Templates$
```

- 5) Execute the following commands:

- \$ls a*n
- \$ls s?
- cat abc>>xyz

Ans.

- i. \$ls a*n

```
pranavv@pranavv:~$ ls a*n
an
pranavv@pranavv:~$
```

- ii. \$ls s?

```
pranavv@pranavv:~$ ls s?  
ls: cannot access 's?': No such file or directory  
pranavv@pranavv:~$
```

iii. cat abc>>xyz

```
pranavv@pranavv:~/p6$ cat abc  
cat: abc: No such file or directory  
pranavv@pranavv:~/p6$ cat abc.txt  
hello  
pranavv@pranavv:~/p6$ cat xyz.txt  
good night  
pranavv@pranavv:~/p6$ cat abc>>xyz  
cat: abc: No such file or directory  
pranavv@pranavv:~/p6$ cat abc.txt>>xyz.txt  
pranavv@pranavv:~/p6$ cat abc  
cat: abc: No such file or directory  
pranavv@pranavv:~/p6$ cat abc.txt  
hello  
pranavv@pranavv:~/p6$ cat xyz.txt  
good night  
hello  
pranavv@pranavv:~/p6$
```

CONCLUSION : We successfully implemented file and directory manipulation commands like ls, mv, cp, rm, join, split, cp, head, tail, touch.

Practical 7 : Execute File and Directory Manipulation commands

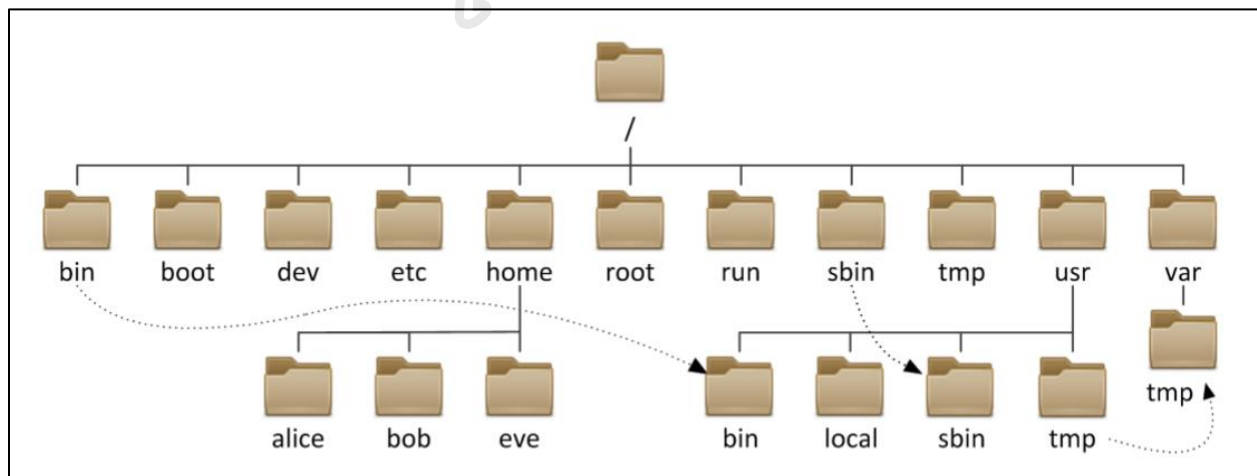
PRACTICAL SIGNIFICANCE :

Data and programs are stored in Files. These are organized in directories. In a simple way, a directory is just a file that contains other files(or directories).

MINIMUM THEORITICAL BACKGROUND :

Unix uses a hierarchical structure for organizing files and Directories. This structure is called as a directory tree. The tree has a single root node, the slash character (/), and all other directories are contained below it. When user first log in to the Unix server, the specified directory is called as Home Directory.

Directory structure in Unix/Linux:



Directory Manipulation commands are as follows :

- a. mdkir
- b. cd
- c. rmdir
- d. pwd

1) **mkdir Command**

It is used to create a new directory in a current directory .

\$ mkdir <directory Name>

Example : \$ mkdir CM5I

User can create more than one directory in a single command .

\$mkdir subject1 subject2

If user wants to create directory on the specific path then syntax is:

Syntax:

\$mkdir path/ directory name

2) **cd Command**

The Cd command is used to change directory. You can use it to change to any directory by specifying a valid absolute or relative path.

Syntax:

\$cd <directory Name>

Example:

\$cd Directory Name

\$cd CM5I

\$cd To come out from current working directory.

Example :

\$cd..

\$cd/ It changes to root directory.

Example:

\$cd/

3) **rmdir Command**

It is used to Delete/ remove directory. If the parent directory having subdirectories then first all subdirectories will be deleted then the parent directory is deleted.

Syntax: \$rmdir

4) **pwd Command**

Pwd stands for present working directory. This is most used linux command to see the specific Unix directory on which the user is working on.

Syntax: \$pwd

diff command

This command is used to show difference between two text files . It also tells which line in one has to be changed to make the two files identical .

Syntax:

`$diff filename1 filename2`

The operations of the result should be like this –

- a - Added the text to file
- c - changes are made in the file
- d - deletion operation is performed
- < - lines from the first file
- > - lines from the second file

\$cat file1.txt

I need to go to the shop
I need to buy some mangoes
When I get home , I'll wash the cat

\$cat file12.txt

I need to go to the shop
I need to buy some mangoes
Oh yeah , I also need to by cheese
When I get home, I'll wash the cat
Use the diff command to compare both files.

\$diff file1.txt file2.txt

The above command should give the result as shown below-

2a3

>oh yeah, I also need to by cheese

From the output 2a3 means “After line 2 in the first file, a line to be added: line 3 from the second file “.

Comm command

This command compare two sorted files. It compares each line of first file with its corresponding line in the second file and generates 3 column output.

The first column lists the lines only in first file.

The second column lists the lines only in second file.

The third column lists the lines in both files.

Syntax:

`$comm filename1filename2`

Example:

<code>\$Cat student1</code>	<code>\$cat student2</code>
Harsh	Harsh

Sujay	Niket	
Smith	Ashutosh	
\$comm student1 student2		
Sujay	Niket	Harsh
Smith	Ashutosh	

Pr command

Pr- convert text files for printing

The pr command does minor formatting of files on the terminal screen or for a printer. For example , if you have a long list of names in a file, you can format it onscreen into two or more columns.

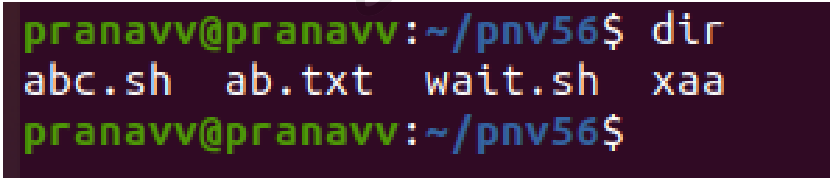
Syntax:-

Pr option(s) filename(s)

Pr changes the format of the file only on the screen or on printed copy ; it doesn't modify the original file.

dir command

It is used to list all files and directories



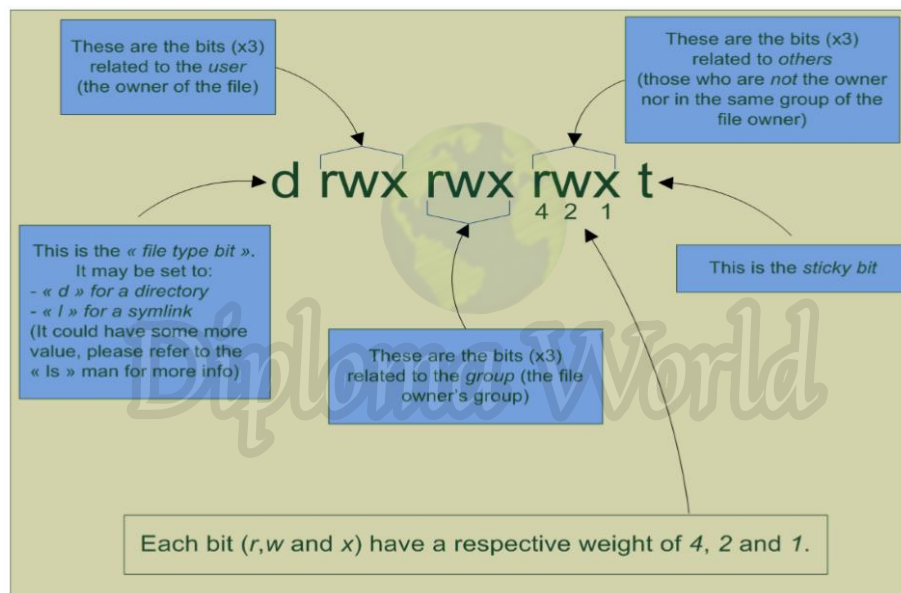
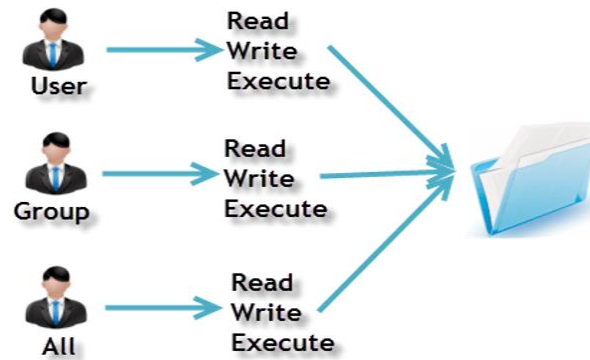
```
pranavv@pranavv:~/pnv56$ dir
abc.sh  ab.txt  wait.sh  xaa
pranavv@pranavv:~/pnv56$
```

chmod command

It is used to change or set permissions (read, write and execute) for all the three categories of users .

Types of users are – user , group and other

Owners assigned Permission On Every File and Directory



To verify The permissions of a file or directory use `ls-l` command.

Syntax:

`$ls -l`

Chmod can be operated in two ways:

1. Symbolic or alphabetical Notation.
2. Octal or absolute Notation.

1) Symbolic or Alphabetical Notation

It uses symbols for assigning permissions to the users. There are following symbols for users of three categories

Option	Symbols	Meaning
Category	u	Owner of the file and directories.
	g	Members of the group belonging to the user.
	o	All other system users.
	a	All the users
Permission	r	r to assign read permission to a file
	w	w to assign write permission to a file
	x	x to assign execute permissions to a file
Operations	+	Assigns the permission
	-	Removes the permission.
	=	Assigns absolute permissions.

Syntax:

\$Chmod<category><operation><permissions><filename>

Example:

- 1) Assign the execute permission for user/owner,group and others.
\$chmod ugo+x abc
- 2) Assign all permissions for group
\$chmod g+rwX abc
- 3) Assign execute permission for the owner and remove read permission of group and others.
\$chmod u+x, go-r abc
- 4) Assign read the permission to all three users and remove all remaining permissions.
\$chmod ugo=r abc

Number	Assignment
0	No permission
1	Execute permission
2	Write permission
4	Read permission
5(4+1)	Read and execute permission
6(4+2)	Read and write permission
7(4+2+1)	Read, write and execute permission

2) Octal and absolute Notation

This method uses a number to specify each set of permissions for the file . It assign permission in three digits . First digit assign permission for owner, second digit for group and third for others . Digits range is 0 to 7.

Syntax:

`$chmod<three digit octal number><filename>`

`$chmod [u g o] file name`

List of octal numbers:

Example: `$chmod 750 abc`

It assign all permissions for user , read and execute for group and no permissions for other for file abc.

 cmp command

This command is used to compare files when used without options . It uses two filename as an argument and display the difference on the terminals.

`$cmp filename1 filename 2`

`$cat student1          $cat student2`

Harsh

Harsh

Sujay

sujay

Smith

smith

`$cmp filename1 filename2`

Student1 student2 diff : char 6 line 1

PROGRAM CODE :

1) Execute File and Directory Manipulation commands.

1. mkdir

```
pranavv@pranavv:~$ mkdir panuv
pranavv@pranavv:~$ ls
a  an  Desktop  Documents  Downloads  Music  n  p6  panuv
```

2. cd

```
pranavv@pranavv:~/pnv56$ cd
pranavv@pranavv:~$ dir
```

3. rmdir

```
pranavv@pranavv:~/pnv56$ ls
abc.sh ab.txt directory wait.sh xaa
pranavv@pranavv:~/pnv56$ rmdir directory
pranavv@pranavv:~/pnv56$ ls
abc.sh ab.txt wait.sh xaa
pranavv@pranavv:~/pnv56$
```

4. pwd

```
pranavv@pranavv:~/pnv56$ pwd
/home/pranavv/pnv56
pranavv@pranavv:~/pnv56$
```

5. diff command

```
pranavv@pranavv:~/p6/p61$ cat a.txt
1 abc
2 pqr
3 xyz
4 pranavibhute

pranavv@pranavv:~/p6/p61$ cat b.txt
1 9112
2 9113
3 9114
4 9115

pranavv@pranavv:~/p6/p61$ diff a.txt b.txt
1,4c1,4
< 1 abc
< 2 pqr
< 3 xyz
< 4 pranavibhute
---
> 1 9112
> 2 9113
> 3 9114
> 4 9115
pranavv@pranavv:~/p6/p61$ diff b.txt a.txt
1,4c1,4
< 1 9112
< 2 9113
< 3 9114
< 4 9115
---
> 1 abc
> 2 pqr
> 3 xyz
> 4 pranavibhute
pranavv@pranavv:~/p6/p61$
```

6. comm command

```

pranavv@pranavv:~/p6/p61$ cat a.txt
1 abc
2 pqr
3 xyz
4 pranavibhute

pranavv@pranavv:~/p6/p61$ cat b.txt
1 9112
2 9113
3 9114
4 9115

pranavv@pranavv:~/p6/p61$ comm a.txt b.txt
      1 9112
1 abc      2 9113
2 pqr      3 9114
3 xyz      4 9115
comm: file 2 is not in sorted order

4 pranavibhute
comm: file 1 is not in sorted order

pranavv@pranavv:~/p6/p61$

```

7. ls -l

```

pranavv@pranavv:~$ ls -l
total 44
-rw-rw-r-- 1 pranavv pranavv  0 Jan 26 21:06 a
-rw-rw-r-- 1 pranavv pranavv  0 Jan 26 21:07 an
drwxr-xr-x 2 pranavv pranavv 4096 Dec 26 20:35 Desktop
drwxr-xr-x 2 pranavv pranavv 4096 Jan 26 20:52 Documents
drwxr-xr-x 2 pranavv pranavv 4096 Dec 26 20:35 Downloads
drwxr-xr-x 2 pranavv pranavv 4096 Dec 26 20:35 Music
-rw-rw-r-- 1 pranavv pranavv  0 Jan 26 21:06 n
drwxrwxr-x 13 pranavv pranavv 4096 Jan 26 21:13 p6
drwxr-xr-x 6 pranavv pranavv 4096 Jan 27 15:43 Pictures
drwxrwxr-x 2 pranavv pranavv 4096 Jan 27 16:19 pnv56
drwxr-xr-x 2 pranavv pranavv 4096 Dec 26 20:35 Public
drwxr-xr-x 2 pranavv pranavv 4096 Jan 26 21:05 Templates
-rw-rw-r-- 1 pranavv pranavv 358 Jan  7 19:01 typescript
drwxr-xr-x 2 pranavv pranavv 4096 Dec 26 20:35 Videos
pranavv@pranavv:~$

```

8. pr command

```

pranavv@pranavv:~/pnv56$ pr ab.txt

2021-01-27 12:20                                ab.txt                                Page 1

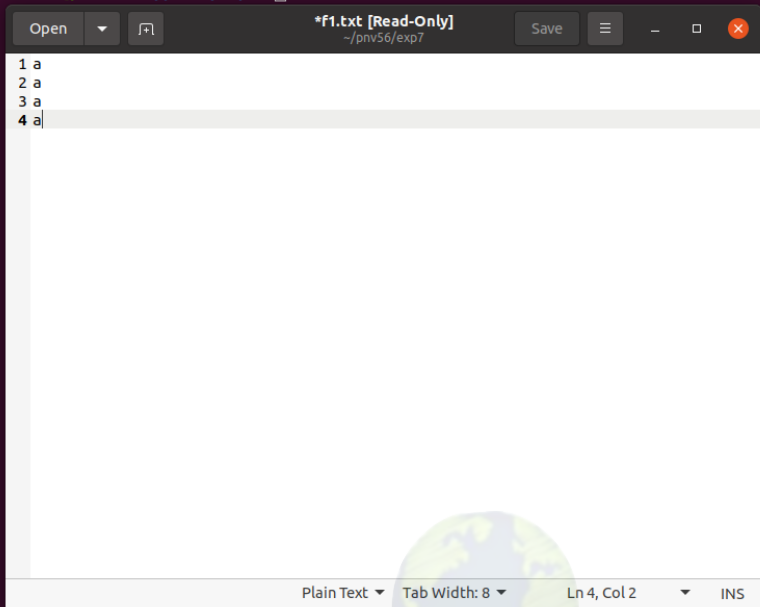
hello everyone

```

9. chmod command

a. Symbolic

```
pranavv@pranavv:~/pnv56/exp7$ chmod ugo-wx f1.txt
pranavv@pranavv:~/pnv56/exp7$
```

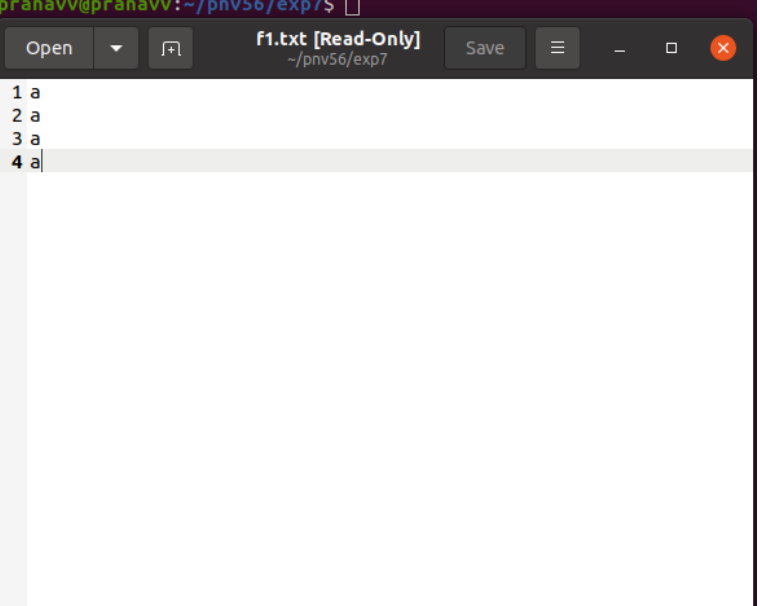


The screenshot shows a text editor window titled "f1.txt [Read-Only]" with the path "~/pnv56/exp7". The editor contains four lines of text, each starting with a line number followed by the letter 'a':
1 a
2 a
3 a
4 a|
The status bar at the bottom indicates "Plain Text", "Tab Width: 8", "Ln 4, Col 2", and "INS".

```
pranavv@pranavv:~/pnv56/exp7$ chmod ugo-wx f1.txt
pranavv@pranavv:~/pnv56/exp7$ chmod ugo+wx f1.txt
pranavv@pranavv:~/pnv56/exp7$ chmod u+rw f1.txt
pranavv@pranavv:~/pnv56/exp7$ chmod o-rwx f1.txt
pranavv@pranavv:~/pnv56/exp7$ chmod g-x f1.txt
pranavv@pranavv:~/pnv56/exp7$ chmod ugo=r f1.txt
pranavv@pranavv:~/pnv56/exp7$
```

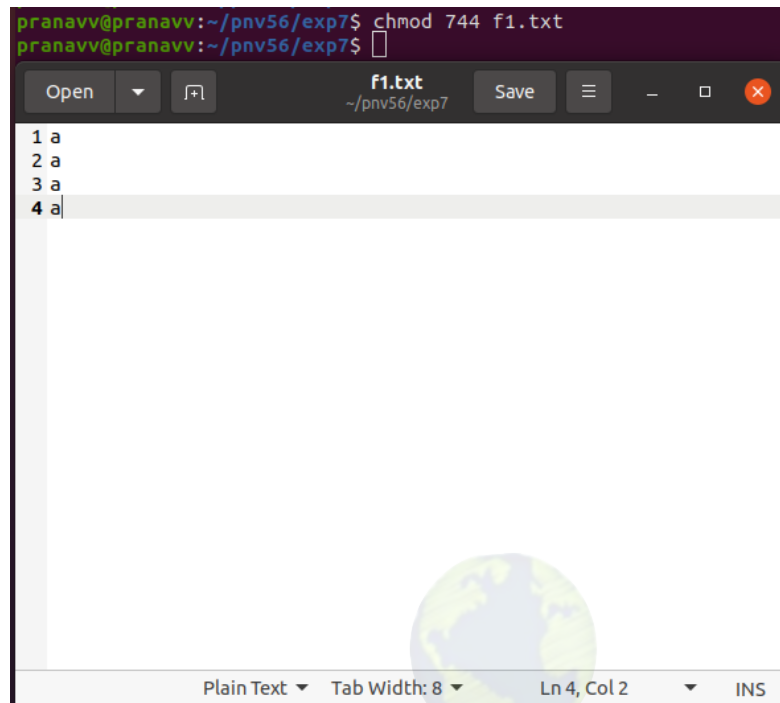
b. Octal

```
pranavv@pranavv:~/pnv56/exp7$ chmod 400 f1.txt
pranavv@pranavv:~/pnv56/exp7$
```



The screenshot shows a text editor window titled "f1.txt [Read-Only]" with the path "~/pnv56/exp7". The editor contains four lines of text, each starting with a line number followed by the letter 'a':
1 a
2 a
3 a
4 a|
The status bar at the bottom indicates "Plain Text", "Tab Width: 8", "Ln 4, Col 2", and "INS".

```
pranavv@pranavv:~/pnv56/exp7$ chmod 744 f1.txt
pranavv@pranavv:~/pnv56/exp7$
```

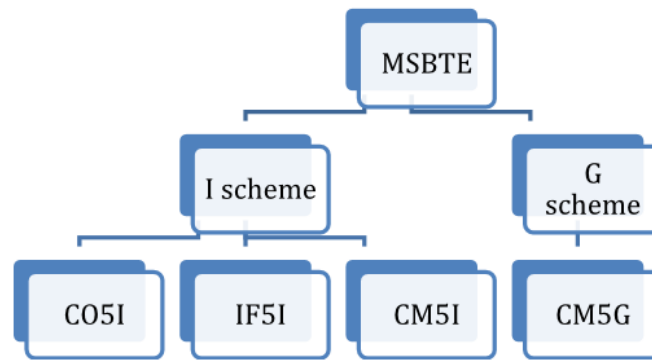


The screenshot shows a text editor window titled 'f1.txt' with the path '~/pnv56/exp7'. The editor contains four lines of text, each starting with a line number followed by a space and the letter 'a':
1 a
2 a
3 a
4 a|
The cursor is at the end of the fourth line. The status bar at the bottom indicates 'Plain Text', 'Tab Width: 8', 'Ln 4, Col 2', and 'INS'.

10. cmp command

```
pranavv@pranavv:~/pnv56/exp7$ cat pqr.txt
l
m
n
o
p
q
r
s
pranavv@pranavv:~/pnv56/exp7$ cat prql.txt
a
b
c
d
e
f
g
pranavv@pranavv:~/pnv56/exp7$ cmp pqr.txt prql.txt
pqr.txt prql.txt differ: byte 1, line 1
pranavv@pranavv:~/pnv56/exp7$ cmp prql.txt pqr.txt
prql.txt pqr.txt differ: byte 1, line 1
pranavv@pranavv:~/pnv56/exp7$
```

2) Create the following structure



```
pranavv@pranavv:~$ mkdir MSBTE
pranavv@pranavv:~$ cd MSBTE
pranavv@pranavv:~/MSBTE$ mkdir IScheme GScheme
pranavv@pranavv:~/MSBTE$ cd IScheme
pranavv@pranavv:~/MSBTE/IScheme$ mkdir C05I IF5I CM5I
pranavv@pranavv:~/MSBTE/IScheme$ cd
pranavv@pranavv:~$ cd MSBTE
pranavv@pranavv:~/MSBTE$ cd GScheme
pranavv@pranavv:~/MSBTE/GScheme$ mkdir CM5G
pranavv@pranavv:~/MSBTE/GScheme$
```

```
pranavv@pranavv:~$ cd MSBTE
pranavv@pranavv:~/MSBTE$ ls
GScheme  IScheme
pranavv@pranavv:~/MSBTE$ cd GScheme
pranavv@pranavv:~/MSBTE/GScheme$ ls
CM5G
pranavv@pranavv:~/MSBTE/GScheme$ cd CM5G
pranavv@pranavv:~/MSBTE/GScheme/CM5G$ cd ..
pranavv@pranavv:~/MSBTE/GScheme$ cd ..
pranavv@pranavv:~/MSBTE$ cd IScheme
pranavv@pranavv:~/MSBTE/IScheme$ ls
CM5I  C05I  IF5I
pranavv@pranavv:~/MSBTE/IScheme$ cd CM5I
pranavv@pranavv:~/MSBTE/IScheme/CM5I$
```

PRACTICAL RELATED QUESTIONS :

1) How to shift from Root directory to User(Home) directory ?

Ans. We can shift from Root Directory to User(Home) directory by using : cd

```
pranavv@pranavv:~$ cd Desktop
pranavv@pranavv:~/Desktop$
```

2) How to see directories?

Ans. We can see directories using: `dir`

```
pranavv@pranavv:~$ dir
a  an  Desktop  Documents  Downloads  MSBTE  Music  n  p6  Pictures  pnv56  Public  Templates  typescript  Videos
pranavv@pranavv:~$
```

3) What is default set of permissions given by syetm to the directory?

Ans. Default permission for a directory is 0777, for files the permissions are 0666 from which the default umask value 0022 is deducted to get the newly created files or directory permission.

4) Assign all the permissions to your directory for all users using symbolic & octal methods.

Ans.

```
pranavv@pranavv:~/Desktop$ chmod a+rwX msbte
pranavv@pranavv:~/Desktop$ chmod 777 msbte
pranavv@pranavv:~/Desktop$
```

5) What is difference between comm and cmp command?

Ans.

Diff Command :

This command is used to show difference between two text files. It also tells which line in one has to be changed to make the two files identical.

Syntax:

`$diff filename1 filename2`

The operations of the result should be like this –

a - Added the text to file

c - changes are made in the file

d - deletion operation is performed

< - lines from the first file

> - lines from the second file

Comm command :

This command compares two sorted files. It compares each line of first file with its corresponding line in the second file and generates 3 column output.

The first column lists the lines only in first file.

The second column lists the lines only in second file.

The third column lists the lines in both files.

Syntax:

\$comm filename1filename2

EXERCISE :

1. Write the commands for performing the following tasks sequentially

a. Display your current directory

```
pranavv@pranavv:~/Desktop$ pwd
/home/pranavv/Desktop
pranavv@pranavv:~/Desktop$
```

b. Create a directory 'subjects' in current directory

```
pranavv@pranavv:~/Desktop$ mkdir subjects
pranavv@pranavv:~/Desktop$ ls
nsbte  subjects
pranavv@pranavv:~/Desktop$
```

c. Create a file 'sample' in the directory 'subject'

```
pranavv@pranavv:~/Desktop$ cd subjects
pranavv@pranavv:~/Desktop/subjects$ >sample.txt
pranavv@pranavv:~/Desktop/subjects$ ls
sample.txt
pranavv@pranavv:~/Desktop/subjects$
```

d. Remove the write permission for the owner for 'sample' using symbolic method

```
pranavv@pranavv:~/Desktop/subjects$ chmod u-w sample.txt
pranavv@pranavv:~/Desktop/subjects$
```

e. Delete the file 'sample'. What is an error message displayed

No error displayed

```
pranavv@pranavv:~/Desktop/subjects$ rm sample.txt
rm: remove write-protected regular empty file 'sample.txt'? Yes
pranavv@pranavv:~/Desktop/subjects$ ls
pranavv@pranavv:~/Desktop/subjects$
```

2. What are the permissions assigned to file/files after execution of following commands?

Ans.

a. \$chmod 700 abc

It assign all permissions for owner and no permissions for group & other for file abc.

b. \$chmod u+rw,go-rwx file1 file2

It assign read, write, execute permissions for owner for file file1 and it removes read, write, execute permissions for group & owner for file file2.

c. \$chmod 536 xyz

It assign read and execute permissions for owner, write and execute permissions for group and write and execute permissions for other for file xyz.

3. Create new files pqr & prql

Perform the commands

a. \$chmod ugo=r pqr

Assign read the permission to all three users and removes all remaining permissions for file pqr.

b. \$chmod ugo+r prql

Assign read the permission to owner, group and other for file prql.

```
pranavv@pranavv:~/pnv56/exp7$ >pqr.txt
pranavv@pranavv:~/pnv56/exp7$ >prql.txt
pranavv@pranavv:~/pnv56/exp7$ ls
pqr.txt  prql.txt
pranavv@pranavv:~/pnv56/exp7$ chmod ugo=r pqr.txt
pranavv@pranavv:~/pnv56/exp7$ chmod ugo+r prql.txt
pranavv@pranavv:~/pnv56/exp7$ ls
pqr.txt  prql.txt
```

4. Assign read and write permission for the owner, write permission for the group and execute permission for others using octal method for file mfile.

Ans.

```
pranavv@pranavv:~/pnv56/exp7$ >mfile.txt
pranavv@pranavv:~/pnv56/exp7$ chmod 621 mfile.txt
pranavv@pranavv:~/pnv56/exp7$ █
```

5. Write commands to assign following permissions to file OSY using octal method

- a. _____
- b. _rw_r_xr__
- c. _r_xr_xr_x

Ans.

- a. \$chmod 000 OSY.txt
- b. \$chmod 654 OSY.txt
- c. \$chmod 555 OSY.txt

6. Write commands to assign following permissions to file OSY using symbolic method

- a. _rwxr_xr__
- b. _rwxrwxrwx

Ans.

- a. \$chmod u+rwx,g+rx,o+r OSY.txt
- b. \$chmod ugo+rwx

CONCLUSION : We successfully implemented file and directory manipulation commands like mkdir, rmdir, cd, pwd ,dir, comm, diff, cmp, chmod etc.

Practical 8 : Execute text processing commands

PRACTICAL SIGNIFICANCE :

There are many text processing commands. A filter is commands that takes flow of data from the standard input, process or filters it and sends the result to standard output. cut, wc, tr, sort or grep are all simple filters.

MINIMUM THEORETICAL BACKGROUND :

1. **tr (Unix):-** **tr** is a command in Unix-like operating systems. It is an abbreviation of **translate or transliterate**, indicating its operation of replacing or removing specific characters in its input data set.

Syntax

The syntax of tr command is:

```
$ tr [OPTION] SET1 [SET2]
```

Translation

If both the SET1 and SET2 are specified and '-d' OPTION is not specified, then tr command will replace each character in SET1 with each character in same position in SET2.

1. Convert lower case to upper case

- The following tr command is used to convert the lower case to upper case
\$ tr
operating system
OPERATING SYSTEM
fifth semester
FIFTH SEMESTER
- The following command will also convert lower case to upper case
\$ tr [:lower:] [:upper:]
java
JAVA
- You can also use ranges in tr. The following command uses ranges to convert lower to upper case.
\$ tr a-z A-Z'

2. wc

The wc (word count) command in Unix/Linux operating systems is used to find out number of lines, words and characters in a file.

The syntax of wc command as shown below:

```
$wc [-l -w -c] <filename>
```

-l counts only number of lines

-w counts only number of words

-c counts only number of characters

Example:

```
$cat student1
```

```
Ajay
```

```
Vijay
```

```
Sujit
```

```
$wc student1
```

```
3 3 14
```

Means 3 lines, 3 words and 14 characters.

```
# wc [options] filenames
```

```
[root@tecmint ~]# wc tecmint.txt
```

```
12 16 112 tecmint.txt
```

The following are the options and usage provided by the command.

Sr. No.	Command	Description
1	wc-l	Prints the number of lines in a file.
2	wc-w	Prints the number of words in a file.
3	wc-c	Displays the count of bytes in a file.
4	wc-m	Prints the count of characters from a file.
5	wc-L	Prints only the length of the longest line

3. Cut

Cut command:

It is used to copy the specified columns to the standard output file. It is used to cut portion of a file. It takes filenames as command line arguments or input from standard input. It does not delete the selected portion of a file.

Syntax:

```
$cut [-f/-c] [n1,n2] filename
```

Where n1,n2 are the field/character number.

-f Displays specified field. Also uses -d for field separator.

-c Displays specified column character by character.

Example 1:

```
$cat student1
Ajay      1990  50
Vijay     1992  60
Sujit     1991  70
$cut -f1,2 student1
Ajay      1990
Vijay     1992
Sujit     1991
```

In this output it will display 1st and 2nd fields values

Example 2:

```
$cat numbers
12345
23123
45656
76543
$cut -c4,5 student2
45
23
56
43
```

In this output it displays 4th and 5th character from each line.

4. Paste

Paste command is one of the useful commands in Unix or Linux operating system. It is used to join files horizontally (parallel merging) by outputting lines consisting of lines from each file specified, separated by tab as delimiter, to the standard output. When no file is specified, or put dash ("-") instead of file name, paste reads from standard input and gives output as it is until a interrupt command [Ctrl-c] is given.

Syntax:

```
paste [OPTION]... [FILES]...
```

Create files for demonstration:

```
$cat stdname
Ajay
Vijay
Sujit
$cat birthyear
1980
2001
```

```
1996
$paste stdname birthyaer
Ajay  1980
Vijay 2001
Sujit 1996
```

5. Spell

Spell is a spell-checking program which scans a text file for misspelled words and prints each misspelled word on its own line.

Syntax:

spell [OPTIONS] FILE ...

-n Print line numbers.

(For each misspelled word, also print the number of the line it was in.)

-o Print file names.

(For each misspelled word, also print the name of the file it was in.)

spell -on words.txt words2.txt

Checks the spelling of files words.txt and words2.txt, printing the file name (-o) and the line number (-n) of each misspelled word.

Jaishree@DESKTOP-OKO5FND ~

\$ cat student1

```
Sujay      2001
Bhavesh    2000
Pinank     2000
```

Jaishree@DESKTOP-OKO5FND ~

\$ sort student1

```
Bhavesh    2000
Pinank     2000
Sujay      2001
```

Jaishree@DESKTOP-OKO5FND ~

\$ cat numbers

```
12
45
2
34
8
```

Jaishree@DESKTOP-OKO5FND ~

\$ sort -n numbers

```
2
8
12
34
45
```

6. Sort

SORT command is used to sort a file,

arranging the records in a particular order. By default, the sort command sorts file assuming the contents are ASCII. Using options in sort command, it can also be used to sort numerically.

When we have a mix file with both uppercase and lowercase letters then first the lower case letters would be sorted following with the upper case letters .

Option	Meaning
-o	write the output to a new file
-n	sort a file numerically
-r	Sort in Reverse Order

7. **grep (Global Regular expression printer)**

It is used to search a pattern/word inside files.

Syntax:

`$grep <option> <pattern> <filename>`

Options	Meaning
---------	---------

-c	Displays the count of the number of occurrences of the pattern
----	--

-l	Displays the list of the filenames which contains a pattern.
----	--

-n	Displays line numbers along with the lines containing a pattern.
----	--

`$cat OS`

Unix

Linux

Ubuntu

`$grep -n Unix OS`

It searches Unix pattern and displays line number where that pattern matched.

8. **more**

If the file is too large for its contents to fit in one screen, it will scroll off your screen when the file is displayed with cat command then use more command.

Syntax:

`$more ["Space bar" /f/b/j/k/200G/G] filename`

Options	Meaning
---------	---------

[Space]	scrolls the display, one screenful of data at a time
---------	--

j	scrolls forward one line
---	--------------------------

f	Scrolls forward one screen
---	----------------------------

b	scrolls backwards one screen
---	------------------------------

k	scrolls backward one line
---	---------------------------

200G	Goes to line number 200
------	-------------------------

G	Goes to last line of the file
---	-------------------------------

PROGRAM CODE :

Execute text processing commands tr, cut, paste, wc, spell ,sort, grep, more

1) tr

```
pranavv@pranavv:~/pnv56/exp8$ cat spell.txt
I studie in Government Polytechnic Miraj
I live in Islampur
pranavv@pranavv:~/pnv56/exp8$ tr "[:lower:]" "[:upper:]" < spell.txt
I STUDIE IN GOVERNMENT POLYTECHNIC MIRAJ
I LIVE IN ISLAMPUR
pranavv@pranavv:~/pnv56/exp8$ tr -d "Islampur" < spell.txt
tdie in Govenent Poytechnic Mij
ive in
pranavv@pranavv:~/pnv56/exp8$
```

2) cut

```
pranavv@pranavv:~/pnv56/exp8$ cat name.txt
1      pranav vibhute
2      arshad shirgave
3      parth shaha
4      parshwa shaha
5      pranav mane
pranavv@pranavv:~/pnv56/exp8$ cut -c 1,10 name.txt
1v
2s
3h
4
5m
pranavv@pranavv:~/pnv56/exp8$
```

3) paste

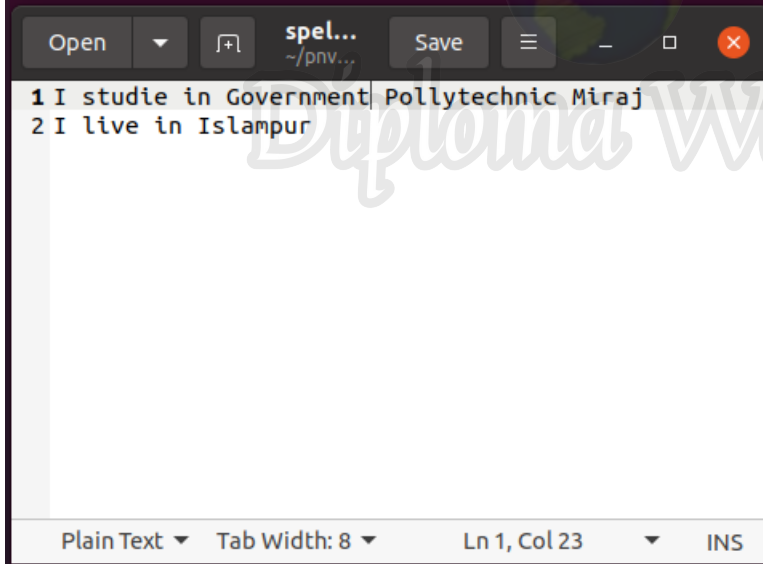
```
pranavv@pranavv:~/pnv56/exp8$ cat name.txt
1      pranav vibhute
2      arshad shirgave
3      parth shaha
4      parshwa shaha
5      pranav mane
pranavv@pranavv:~/pnv56/exp8$ cat roll.txt
1      18356
2      18352
3      18348
4      18349
5      18332
pranavv@pranavv:~/pnv56/exp8$ paste name.txt roll.txt
1      pranav vibhute 1      18356
2      arshad shirgave 2      18352
3      parth shaha    3      18348
4      parshwa shaha  4      18349
5      pranav mane    5      18332
pranavv@pranavv:~/pnv56/exp8$
```

4) wc

```
pranavv@pranavv:~/pnv56/exp8$ cat name.txt
1   pranav vibhute
2   arshad shirgave
3   parth shaha
4   parshwa shaha
5   pranav mane
pranavv@pranavv:~/pnv56/exp8$ wc name.txt
 5 15 79 name.txt
pranavv@pranavv:~/pnv56/exp8$ wc -c name.txt
79 name.txt
pranavv@pranavv:~/pnv56/exp8$ wc -l name.txt
5 name.txt
pranavv@pranavv:~/pnv56/exp8$ wc -w name.txt
15 name.txt
pranavv@pranavv:~/pnv56/exp8$
```

5) spell

```
pranavv@pranavv:~/pnv56/exp8$ spell spell.txt
studie
Pollytechnic
Miraj
Islampur
pranavv@pranavv:~/pnv56/exp8$
```



6) sort

```
pranavv@pranavv:~/pnv56/exp8$ cat table.txt
4 - parth
3 - parshwa
1 - pranav
2 - arshad
pranavv@pranavv:~/pnv56/exp8$ sort table.txt
1 - pranav
2 - arshad
3 - parshwa
4 - parth
pranavv@pranavv:~/pnv56/exp8$ sort -k 3 table.txt
2 - arshad
3 - parshwa
4 - parth
1 - pranav
pranavv@pranavv:~/pnv56/exp8$
```

7) grep

```
pranavv@pranavv:~/pnv56/exp8$ grep -c name spell.txt
0
pranavv@pranavv:~/pnv56/exp8$ grep -c name spell.txt table.txt
spell.txt:0
table.txt:0
pranavv@pranavv:~/pnv56/exp8$ grep -c name spell.txt table.txt name.txt
spell.txt:0
table.txt:0
name.txt:0
pranavv@pranavv:~/pnv56/exp8$
```

8) more

```
pranavv@pranavv:~/pnv56/exp8$ more more.txt
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
1  
2  
3  
4  
--More--(87%)
```

PRACTICAL RELATED QUESTIONS :

- 1) Give the applications of paste commands

Ans. Paste command is one of the useful commands in Unix or Linux operating system. It is used to join files horizontally (parallel merging) by outputting lines consisting of lines from each file specified, separated by **tab** as delimiter, to the standard output.

2) How to move cursor at the end of line

Ans. Ctrl+E or End command can be used moves the cursor to the end of the line.

3) What are options of wc commands

Ans.

1. **-l**: This option prints the **number of lines** present in a file. With this option wc command displays two-columnar output, 1st column shows number of lines present in a file and 2nd itself represent the file name.
2. **-w**: This option prints the **number of words** present in a file. With this option wc command displays two-columnar output, 1st column shows number of words present in a file and 2nd is the file name.
3. **-c**: This option displays **count of bytes** present in a file. With this option it display two-columnar output, 1st column shows number of bytes present in a file and 2nd is the file name.
4. **-m**: Using **-m** option 'wc' command displays **count of characters** from a file
5. **-L**: The 'wc' command allow an argument **-L**, it can be used to print out the length of longest (number of characters) line in a file. So, we have the longest character line *Arunachal Pradesh* in a file **state.txt** and *Hyderabad* in the file **capital.txt**. But with this option if more than one file name is specified then the last row i.e. the extra row, doesn't display total but it display the maximum of all values displaying in the first column of individual files
6. **-version**: This option is used to display the version of **wc** which is currently running on your system.

4) What are different types of filters used in Linux

Ans.

- **cat** : Displays the text of the file line by line.
- **head** : Displays the first n lines of the specified text files. If the number of lines is not specified then by default prints first 10 lines
- **tail** : It works the same way as head, just in reverse order. The only difference in tail is, it returns the lines from bottom to up.

- **sort** : Sorts the lines alphabetically by default but there are many options available to modify the sorting mechanism. Be sure to check out the man page to see everything it can do
- **uniq** : Removes duplicate lines. uniq has a limitation that it can only remove continuous duplicate lines(although this can be fixed by the use of piping). Assuming we have the following data.
- **wc** : wc command gives the number of lines, words and characters in the data

5) What is difference between \$cat abc and \$cat abc more

Ans. Cat displays file contents. If the file is large the contents scroll off the screen before we view it. So command 'more' is like a pager which displays the contents page by page.

EXERCISE :

1) Write the significance of following

I. Only one character is specified

\$ paste -d "|" number state capital

1|Arunachal Pradesh|Itanagar

2|Assam|Dispur

3|Andhra Pradesh|Hyderabad

4|Bihar|Patna

5|Chhattisgrah|Raiपुर

II. More than one character is specified

\$ paste -d "|," number state capital

1|Arunachal Pradesh,Itanagar

2|Assam,Dispur

3|Andhra Pradesh,Hyderabad

4|Bihar,Patna

5|Chhattisgrah,Raiपुर

First and second file is separated by '|' and second and third is separated by ','.After that list is exhausted and reused.

III. -s(serials) ,combination of -d and -s ,-versions (write its syntax and example)

1) -s (serial):

Syntax :

\$ paste -s number state capital

Example

1	2	3	4	5
Arunachal Pradesh	Assam	Andhra Pradesh	Bihar	Chhattisgrah
Itanagar	Dispur	Hyderabad	Patna	Raipur

2) Combination of -d and -s

Syntax :

\$ paste -s -d ":" number state capital

Example

1:2:3:4:5
Arunachal Pradesh:Assam:Andhra Pradesh:Bihar:Chhattisgrah
Itanagar:Dispur:Hyderabad:Patna:Raipu

3) -version

Syntax :

\$ paste -version

Example

paste (GNU coreutils) 8.26
Packaged by Cygwin (8.26-2)
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later .
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

1. Create a new file and practice executing shell commands from within the editor.
Capture the results of some shell commands into file

K command= Move up one line.

Before using k command.

```
We are studying operating systems
B
```

After using k command

```
We are studying operating systems
B
```

X commands: Delete character to the left of the cursor

Before using X command

```
maharashtra state board of technical ed
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
```

After using X command

```
maharashtra state board of technical ed
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

I command= Move right one character

Before using l command.

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

After using l command

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

2. How to get help?

Ans. To get information about a single internal command, reading the entire shell document is not only a time-consuming process but also a tedious task. Luckily, shell offers a command, known as the help command

The help command uses the following syntax.

\$help [options] [text-string/pattern/internal-command]

Example

```

pranavv@pranavv:~/pnv56/exp8$ help cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
Change the shell working directory.

Change the current directory to DIR. The default DIR is the value of the
HOME shell variable.

The variable CDPATH defines the search path for the directory containing
DIR. Alternative directory names in CDPATH are separated by a colon (:).
A null directory name is the same as the current directory. If DIR begins
with a slash (/), then CDPATH is not used.

If the directory is not found, and the shell option 'cdable_vars' is set,
the word is assumed to be a variable name. If that variable has a value,
its value is used for DIR.

Options:
-L      force symbolic links to be followed: resolve symbolic
links in DIR after processing instances of '..'
-P      use the physical directory structure without following
symbolic links: resolve symbolic links in DIR before
processing instances of '..'
-e      if the -P option is supplied, and the current working
directory cannot be determined successfully, exit with
a non-zero status
-@      on systems that support it, present a file with extended
attributes as a directory containing the file attributes

The default is to follow symbolic links, as if '-L' were specified.
'..' is processed by removing the immediately previous pathname component
back to a slash or the beginning of DIR.

Exit Status:
Returns 0 if the directory is changed, and if $PWD is set successfully when
-P is used; non-zero otherwise.
pranavv@pranavv:~/pnv56/exp8$

```

2) Try the commands and write the outputs with its meaning

i. `tr "[a-f]" "[1-5]" < employee`

Output:

```

pranavv@pranavv:~/pnv56/exp8$ cat employee.txt
This is ubuntu terminal
Hi i am pranav
Nice meeting you
pranavv@pranavv:~/pnv56/exp8$ tr "[a-f]" "[1-5]" <employee.txt
This is u2untu t5rmin1l
Hi i 1m pr1n1v
Ni35 m55ting you
pranavv@pranavv:~/pnv56/exp8$

```

Meaning:

It can convert the letters from a-f to 1-5 in employee table

It starts from A=1 to E=5 in that file for conversion

ii. `tr -s " " < employee`

Output:

```
pranavv@pranavv:~/pnv56/exp8$ tr -s "" <employee.txt
This is ubuntu terminal
Hi i am pranav
Nice meeting you
pranavv@pranavv:~/pnv56/exp8$
```

Meaning:

To squeeze repeat occurrences of characters specified in a set use the -s option. This removes repeated instances of a character.

iii. `tr -d "f" < employee`

Output :

```
pranavv@pranavv:~/pnv56/exp8$ tr -d "u" <employee.txt
This is bnt terminal
Hi i am pranav
Nice meeting yo
pranavv@pranavv:~/pnv56/exp8$
```

Meaning:

To delete specific characters use the -d option. This option deletes characters in the first set specified. In that f character deleted from it.

CONCLUSION : We successfully implemented text processing commands to perform operations using commands like tr, cut, paste, spell, sort, grep, more, wc.



Practical 9 : Execute File and Directory Manipulation commands

PRACTICAL SIGNIFICANCE :

There are many ways to edit files in Unix. Editing files using the screen oriented text editor vi in one of the best ways. This editor enables you to edit lines in context with other lines in the file. Vi is intended as a plain text editor (similar to Notepad on Windows, or Textedit on Mac) as opposed to word processing suite such as Word or pages. It does, however have a lot more power compared to Notepad or Textedit.

MINIMUM THEORETICAL BACKGROUND :

Vi modes commands:

Command	Action
K	Move one line upwards

L	Move one character to the right
H	Move one character to the left
W	Move one word to the right
W	Move one word to the right past punctuation
B	Move one word to the left
B	Move one word to the left past punctuation
E	Move to the end of the current word
1G	Move to the beginning of the file
H	Move to the top of the current screen
M	Move to the middle of the current screen
L	Move to the bottom of the current screen
Ctrl-G	Move to the last line in the file
Ctrl-F	Move one screen towards the end of the file
Ctrl-D	Move 1/2 screen towards the end of the file
Ctrl-B	Move one screen towards the beginning of the file
Ctrl-U	Move 1/2 screen towards the beginning of the file
Ctrl-L	Refresh the screen

5G	Move to line 5 of the file (5 can be any line number)
/string	Find text string forward
?string	Find text string backward
N	Find forward next string instance after a string search
N	Find backward next string instance after a string search
ZZ	Save the file exit vi
X	Delete the character at the cursor
X	Delete the character behind the cursor
Dd	Delete the line the cursor is on
10dd	Delete the 10 lines following the cursor
Yy	Yank the current line
P	Put the yanked line below the current line
P	Put the yanked line above the current line

Commands mode commands:

Command	Action
:g/X/s//x/g	Global Search and replace (X=search object x=replace object)

:r file	Import a file into the current file
:34 r file	Import a file into the current file after line 34
:w	Write out the file to save changes
:w file	Write the file to named file
:wq	Save the file exit vi
:w!	Force save the file
:q!	Quit vi but don't save changes

Input mode commands:

Diploma World

Command	Action
A	Insert characters to the right of the cursor
A	Append characters to the current line
I	Insert characters to the left of cursor
I	Insert characters at the beginning of the current line
O	Add a new line after current line
O	Insert a new line above the current line

PROGRAM CODE :

Create a file OS and type at least 30 lines in it

1. Write the command to save the file.

Ans: To save the file and exit at the same time, you can use the ESC and :x key and hit [Enter] . Optionally, press [Esc] and type Shift + Z Z to save and exit the file.

2. Write the command to insert text at the end of line.

Ans: Type A to add text to the end of a line. To see how this command works, position the cursor anywhere on a text line and type A . The cursor moves to the end of the line, where you can type your additions. Press Esc when you are finished.

3. Replace some character into beginning of second line.

Ans: :%s/ plus the text to find, another /, followed by the replacement text/

4. Write a command to delete 10 lines at a time.

Ans: :[start],[end]d
Example= :1,10d

5. Write a command to move cursor 3 words to the right.

Ans: To move right, press l (press 3 time).

Starting the vi Editor

The following table lists out the basic commands to use the vi editor –

Sr. No.	Command	Description
1	vi filename	Creates a new file if it already does not exist, otherwise opens an existing file.

2	vi -R filename	Opens an existing file in the read-only mode.
3	view filename	Opens an existing file in the read-only mode.

Following is an example to create a new file test1 if it already does not exist in the current working directory –

The above command will generate the following output

```
$vi test1
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

```
"test1" [New File]
```



You will notice a tilde (~) on each line following the cursor. A tilde represents an unused line. If a line does not begin with a tilde and appears to be blank, there is a space, tab, newline, or some other non-viewable character present.

Operation Modes

While working with the vi editor, we usually come across the following two modes –

Command mode – This mode enables you to perform administrative tasks such as saving the files, executing the commands, moving the cursor, cutting (yanking) and pasting the lines or words, as well as finding and replacing. In this mode, whatever you type is interpreted as a command.

Insert mode – This mode enables you to insert text into the file. Everything that's typed in

this mode is interpreted as input and placed in the file.

vi always starts in the command mode. To enter text, you must be in the insert mode for which simply type **i**. To come out of the insert mode, press the Esc key, which will take you back to the command mode.

Moving within a File

To move around within a file without affecting your text, you must be in the command mode (press Esc twice).

Sr. No.	Command	Description
1	k	Moves the cursor up one line
2	j	Moves the cursor down one line
3	h	Moves the cursor to the left one character position
4	l	Moves the cursor to the right one character position

Control Commands:-

Editing Files

To edit the file, you need to be in the insert mode. There are many ways to enter the insert mode from the command mode –

Sr. No.	Command	Description
1	i	Inserts text before the current cursor location
2	I	Inserts text at the beginning of the current line
3	a	Inserts text after the current cursor location
4	A	Inserts text at the end of the current line
5	o	Creates a new line for text entry below the cursor location
6	O	Creates a new line for text entry above the cursor location

Page movement Description:-

Sr. No.	Command	Description
1	CTRL+d -	Move forward 1/2 screen
2	CTRL+f-	Move forward one full screen
3	CTRL+u -	Move backward 1/2 screen
4	CTRL+b	Move backward one full screen
5	CTRL+e	Moves screen up one line
6	CTRL+y-	Moves screen down one line
7	CTRL+I	Redraws screen

Deleting Characters

Here is a list of important commands, which can be used to delete characters and lines in an open file –

Sr. No.	Command	Description
1	x	Deletes the character under the cursor location
2	X	Deletes the character before the cursor location
3	Dw	Deletes from the current cursor location to the next word
4	d^	Deletes from the current cursor position to the beginning of the line
5	d\$	Deletes from the current cursor position to the end of the line
6	D	Deletes from the cursor position to the end of the current line
7	dd	Deletes the line the cursor is on

Change Commands

You also have the capability to change characters, words, or lines in vi without deleting them.

Sr. No.	Command	Description
1	cc	Removes the contents of the line, leaving you in insert mode.
2	cw	Changes the word the cursor is on from the cursor to the lowercase w end of the word.
3	r	Replaces the character under the cursor. vi returns to the command mode after the replacement is entered.
4	R	Overwrites multiple characters beginning with the character currently under the cursor. You must use Esc to stop the overwriting.
5	s	Replaces the current character with the character you type. Afterward, you are left in the insert mode.
6	S	Deletes the line the cursor is on and replaces it with the new text. After the new text is entered, vi remains in the insert mode.

Copy and Paste Commands

You can copy lines or words from one place and then you can paste them at another place using the following commands –

Sr. No.	Command	Description
1	yy	Copies the current line.
2	yw-	Copies the current word from the character the lowercase w cursor is on, until the end of the word.
3	p	Puts the copied text after the cursor
4	P	Puts the yanked text before the cursor.

Ex mode command

Press Esc key and then (:) colon to enter ex-mode commands. A colon is displayed at the left hand corner of the last line on your screen.

Command	Action
:w	Saves file and remains in editing mode
:x	Saves file and quits editing mode
:wq	Save and quit
:q	Quits vi when no changes are made
:q!	Quits vi cancelling the changes
:sh	Escape to Unix shell

Advanced Commands

There are some advanced commands that simplify day-to-day editing and allow for more efficient use of vi –

Word and Character Searching

The vi editor has two kinds of searches: **string** and **character**. For a string search, the / and ? commands are used. When you start these commands, the command just typed will be shown on the last line of the screen, where you type the particular string to look for.

These two commands differ only in the direction where the search takes place –

- The / command searches forwards (downwards) in the file.
- The ? command searches backwards (upwards) in the file.

The **n** and **N** commands repeat the previous search command in the same or the opposite direction, respectively. Some characters have special meanings. These characters must be preceded by a backslash (\) to be included as part of the search expression.

Sr. No.	Character	Description
1	^	Searches at the beginning of the line (Use at the beginning of a search expression).
2	.	Matches a single character.

File is created

```
"operating" [New File]
```

K command= Move up one line.

Before using k command.

```
We are studying operating systems
B
```

After using k command

```
We are studying operating systems
B
```

j command = Move down one line

Before using j command.

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
```

After using j command

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
```


l command= Move right one character

Before using `l` command.

```

maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os

~
~
~
~
~
~
~
~
~
~

```

After using l command

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
```

h command= Move left one character

Before using h command

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os

~
~
~
~
~
~
~
~
~
~
```

After using h command.

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

x command= Delete character at the cursor

Before using x command

```
maharashtra state board of technical education
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

After using x command

```
maharashtra state board of technical edu
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

X commands: Delete character to the left of the cursor

Before using X command

```
maharashtra state board of technical edu
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

After using X command

```
maharashtra state board of technical u
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
~
~
```

r command: Replace character at cursor with one other character

Before using r command

```
maharashtra state board of technical u
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
```

After using r command

```
maharashtra state board of technical e
os is an operating system
Types of OS
Batch os
multitasking os
real time os
~
~
~
~
~
~
~
~
```

yy command: Yank or copy line

```
Hii...My Name is Utkarsha Avirat Sutar
~
~
~
~
~
~
~
~
~
~
```

```
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
Hii...My Name is Utkarsha Avirat Sutar
~
~
~
~
~
~
~
~
```

P command: paste line

Before using dd command

After using dd command

[illegible]

```
pranavv@pranavv:~$ vi os
pranavv@pranavv:~$
```

```
hi my name is pranav vibhute
```

q command Quit without saving changes

```
Operating system is a very good subject
```

```
:q
```

PRACTICAL RELATED QUESTIONS :

1. What is difference between Type o and type of O?

Ans:

Sr No	o	O
1	Creates a new line for text	Creates a new line for text entry


```
:set ic
```

Hide line numbers

```
:set nonu
```

Unsets auto indent

and to delete line to the left of cursor.

1. **Command mode:** in this mode, you can open or create files, specify cursor position and editing command, save or quit your work. Press Esc key to return to Command mode.
2. **Insert mode:** In this mode, you can enter text into the display on your screen to edit the file. To enter the Entry mode, you need to specify an input

In visual block mode, you can press I to insert text at the same position in multiple lines, and you can press A to append text to each line in a block. As well as inserting or appending text that you type, you can insert or append text from registers, for example, the clipboard.

iii. delete contents using command

It is important command .Which can be to delete line .We can delete multiple lines at a time using this command.

2. Create a new file and practice executing shell command from which thing the editor capture the result of some shell commands into the file

Ans. script file_nm
Execute commands
Run exit commands
cat file_nm

3. How to get help?

Ans.
a. Open vi editor.
b. Then click on help option.

CONCLUSION : So this practical was all about Vi editor. We have learned to create a new file or open an existing one. we have also learned to insert, delete or remove the words from a text file. In this practical, we run all the commands related to vi editor.

Practical 10 : Execute Shell Script by using if statement

PRACTICAL SIGNIFICANCE :

If is a statement that allows the programmer to make a decision in the program based on conditions he specified. If the condition is met, the program will execute other task the programmer specified.

MINIMUM THEORETICAL BACKGROUND :

The if...else...fi statement is the control statement that allows Shell to execute statement in controlled way and make the right choice.

Syntax:

```
if [ expression ]
then
    Statements(s) to be executed if expression is true
else
    Statements(s) to be executed if expression is not true
fi
```

The shell expression is evaluated in the above syntax. If the resulting value is true, given statement(s) are executed .If the expression in false, then no statement will be executed . Relational operators are used along with if statement.

Relational Operator used by if:

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-ge	Greater than or equal to
-lt	Less than
-le	Less than or equal to

General Syntax:

1. Single Decision :-

Syntax:-

```
if [ condition ]
then
    ### series of code goes here
fi
```

2. Double Decision: -

Syntax:-

```
if [ condition ]
then
    Statements if the condition is satisfied
else
    Statements of code if the condition is not satisfied
fi
```

3. Multiple if Condition :-

Syntax:-

```
if [ condition1 ]
then
    Statement for condition1
elif [ condition2 ]
then
    Statements for condition2
else
    Statements if the condition is not satisfied
fi
```

4. Double –Bracket :-

Syntax:-

```
if ((condition))
then
    Statements goes here
fi
```

To type any program in Linux you need the compiler. vi editor is used for creating file in Linux.

Write a shell script to display Welcome message with today's date in it. In vi editor wright a program code and save and quit with filename'sample.sh'.

```
$vi sample.sh
```

(Now press escape and i to change the input mode and type following program)

```
#sample.sh
```

```
    #use of if-then statement
```

```
    if whoami; then
```

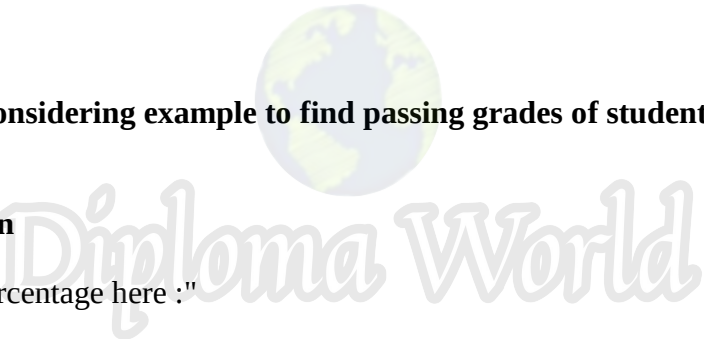
```
varl='whoami'
echo " Welcome $varl"
fi
if date; then
    echo "It display todays date"
fi
echo "This is end of script"
To run the script
$.sample.sh
Save this program press escape+shift+(colon) and type wq (:wq )
"Welcome User!"
Mon Feb 25 22:29:29 PST 2019
"It displays todays date"
"This is end of script"
```

PROGRAM CODE :

Execute shell script by considering example to find passing grades of students using if statement :

1. Single Decision

```
echo "Enter percentage here :"
```



```
read grade

if [ $grade -ge 90 ] && [ $grade -le 100 ]
then
    echo "Grade A"
fi

if [ $grade -ge 70 ] && [ $grade -lt 90 ]
then
    echo "Grade B"
fi

if [ $grade -ge 50 ] && [ $grade -lt 70 ]
then
```

```
echo "Grade C"

fi

if [ $grade -ge 35 ] && [ $grade -lt 50 ]

then

echo "Grade D"

fi

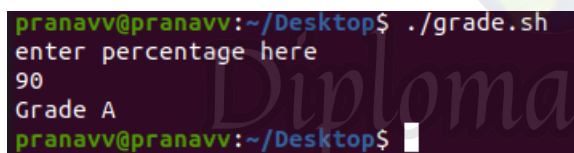
if [ $grade -lt 35 ]

then

echo "FAIL"

fi

Output :
```



```
pranavv@pranavv:~/Desktop$ ./grade.sh
enter percentage here
90
Grade A
pranavv@pranavv:~/Desktop$
```

2. Double Decision

```
echo "Enter percentage of Student : "

read grade

if [ $grade -gt 40 ]

then

echo "Pass"

else

echo "Fail"

fi

Output :
```

```
pranavv@pranavv:~/Desktop$ ./double.sh
Enter percentage of student :
40
Fail
pranavv@pranavv:~/Desktop$ ./double.sh
Enter percentage of student :
60
Pass
pranavv@pranavv:~/Desktop$
```

3. Multiple if statements

```
echo "Enter percentage of Student : "
```

```
read grade
```

```
if [ $grade -ge 90 ]
```

```
then
```

```
echo "Grade A"
```

```
elif [ $grade -ge 80 ]
```

```
then
```

```
echo "Grade b"
```

```
elif [ $grade -ge 70 ]
```

```
then
```

```
echo "Grade c"
```

```
elif [ $grade -ge 60 ]
```

```
then
```

```
echo "Grade d"
```

```
elif [ $grade -ge 40 ]
```

```
then
```

```
echo "Grade e"
```

```
else
```



```
echo "Grade Fail"
```

```
fi
```

Output :

```
pranavv@pranavv:~/Desktop$ ./multiple.sh
Enter percentage of students :
87
Grade B
pranavv@pranavv:~/Desktop$
```

PRACTICAL RELATED QUESTIONS :

1) Write and execute script for nested if statements.

Ans.

```
read -p 'username :' username
```

```
if [ "$username" == "abc" ]
```

```
then
```

```
read -p 'password :' password
```

```
if [ "$password" == "pwd" ]
```

```
then
```

```
echo "Login Successfully"
```

```
else
```

```
echo "Wrong Password"
```

```
fi
```

```
else
```

```
echo "Wrong Username"
```

```
fi
```

Output :

```

utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./login.sh
Username : abc
Password : pwd
Login Successfully
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ █

```

2) Write Difference between

Ans.

If[condition]	If((condition))
The single brackets [...] is the command .	The double parentheses ((...)) is the format for bash arithmetic expansion.
It is used to create commands in statements.	It is used to test an arithmetic operations.
Syntax :- if [condition] then ### series of code goes here fi	Syntax:- if((condition)) then Statements goes here fi
Example :- if ["\$X"-lt"0"] then echo "X is less then zero" fi	Example:- if [[\$num -eq 42]] then # if/then branch echo 'num is actually equal to 42' else # else branch echo 'num is not 42' fi

3) Write script for finding greatest number among given three number.

Ans.

```

echo "Enter number1 number : "
read n1

echo "Enter number2 number : "
read n2

echo "Enter number3 number : "
read n3

```

```
if [ $n1 -gt $n2 ] && [ $n1 -gt $n3 ]
```

```
then
```

```
echo "$n1 is Greater"
```

```
elif [ $n2 -gt $n3 ]
```

```
then
```

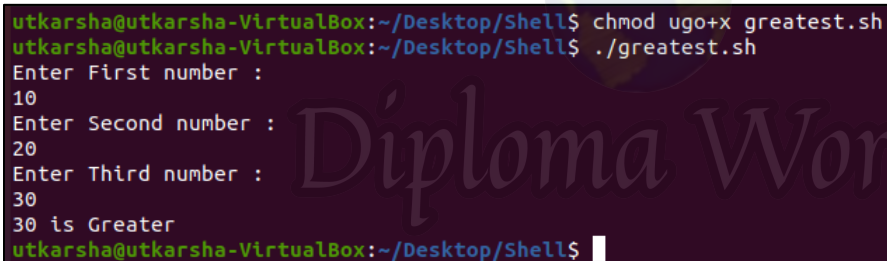
```
echo "$n2 is Greater"
```

```
else
```

```
echo "$n3 is Greater"
```

```
fi
```

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x greatest.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./greatest.sh
Enter First number :
10
Enter Second number :
20
Enter Third number :
30
30 is Greater
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

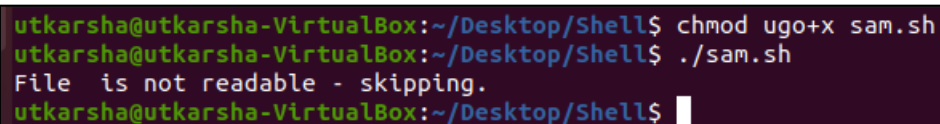
EXERCISE :

1) Correct the following script and write its output.

a. if [! -r "\$l"] then echo " File \$l is not readable – skipping."; fi

Ans.

Error :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x sam.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./sam.sh
File is not readable - skipping.
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

Program code:

```
read l
if [ ! -r $l ]
then
echo "File $l is not readable – skipping."
fi
O
```

```
osboxes@osboxes:~$ ./sam.sh
ex.txt
"File ex.txt is not readable - skipping."
osboxes@osboxes:~$ ./sam.sh
single-if.sh
```

:

- a. if ["\$x" -nt "/etc/passwd"]; Then
echo "X is file which is newer than /etc/passwd"

if

Ans.

Error :

```
osboxes@osboxes:~$ ./sam.sh
./sam.sh: line 5: syntax error: unexpected end of file
```

Program code:

```
read x
if [ $x -nt "/etc/passwd" ]
then
echo "$x is file which is newer than /etc/passwd"
fi
```

Output :

```
osboxes@osboxes:~$ ./sam.sh  
ex.txt  
ex.txt is file which is newer than /etc/passwd
```

CONCLUSION : We successfully executed Shell Script by using if statements (Single decision, Double decision, Multiple if condition)



Practical 11 : Execute Shell Script by using for statements

PRACTICAL SIGNIFICANCE :

The for loops iterate through a set of values until the list is exhausted. In simple terms the for loop operates on lists of items. It repeats a set of commands for every item in a list.

MINIMUM THEORETICAL BACKGROUND :

Syntax of FOR loop :

```
for var in item1 item2...itemN
```

```
do
```

```
command1
```

```
command2
```

```
.... ..
```

```
command
```

```
done
```

Example:

```
For var in 0 1 2 3 4 5 6 7 8 9
```

```
do
```

```
echo $var
```

```
done
```

So far you have looked at creating loops and working with loops to accomplish different tasks. Sometimes you need to stop a loop or skip iterations of the loop. Following two statements that are used to control shell loops.

Three-expression bash for loops syntax

This type of for loop can also be used similar to the C programming language. It is having by a threeparameter loop control expression; consisting of an initializer (EXP1), a loop-test or condition (EXP2), and a counting expression(EXP3).

Three-expression bash for loops syntax

A representative three-expression example in bash as follows:

```
#!/bin/bash
```

```
For((c=1;c<=5;c++)
```

```
do
```

```
echo —Welcome Sc times
```

```
done
```

1. The break statements: The break statement is used to terminate the execution of the entire loop, after completing the execution of all of the lines of code up to the break statement using break keyword .
2. The continue statement: The continue statement is similar to the break command, except that it causes the current iteration of the loop to exit, rather than the entire loop.
3. Exit:

The syntax is as follows:

```
exit N
```

The exit statement is used to exit from the shell script with a status of N.

Write a shell script to calculate the factorial of a given number.

In vi editor write a program code and save and quit with filename `__fact.sh`.

```
$vi fact.sh
```

(Now press escape and i to change the input mode and type following program)

Similar other loops also used in Linux such as while, case..esac

While statement

Syntax:

```
While[[condition]
```

```
do
```

...

...

...

Done

Example:

```
a=1
```

```
while[$a-le 5]
```

```
do
```

```
echo —value of a=$a
```

```
a='expr$a+1'
```

```
done
```

```
case..esac statement;
```

Syntax:

Case value in

```
pattern 1)commands;;
```

```
pattern 2)commands;;
```

```
pattern 3)commands;;
```

```
..
```

```
*)Default
```

```
esac //indication of end of case statement
```

Example:

```
#!/bin/bash
```

```
NOW=$(date+!%a!)
```

```
case $NOW in
```



Mon)

echo —Full backup|;;

Tue|Wed|Thu|Fri)

echo —Partial backup|;;

Sat|Sun)

echo —No backup|;;

*);;

esac

\$cat fast.sh

echo —Enter a number|

read num

fact=1

while[\$num-gt 1]

do

fact=\$((fact*num))

num=\$((num-1))

done

echo —factorial of a numberis \$fact|

Save this program press escape+shift+:(colon)and type wq(:wq)

To compile the program

\$/fact.sh or \$sh fact.sh

Enter the number 5:

Factorial of a given number is 120

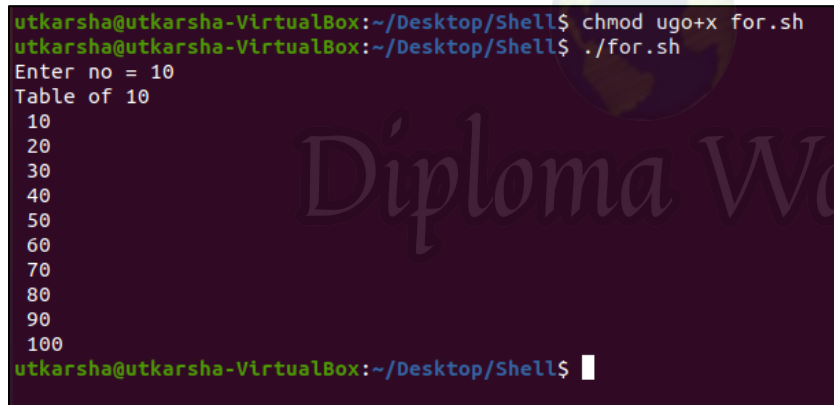
PROGRAM CODE :

- 1) Execute shell script by considering example like printing table of given number by FOR loop

Ans.

```
read -p 'Enter no = ' no
echo "Table of $no"
for (( i=1; i<=10; i++ ))
do
echo " "$(( no * i ))
done
```

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x for.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./for.sh
Enter no = 10
Table of 10
10
20
30
40
50
60
70
80
90
100
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

- 2) Execute shell script by considering example like printing
following output by FOR loop

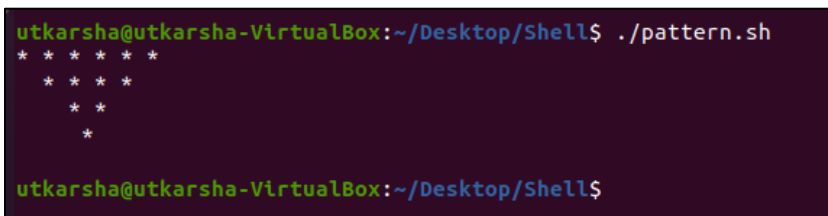
```
*****
*****
**
*
```

Ans.

```
for (( i=1;i<8;i=i+2 ))
```

```
do
for (( j=1;j<7;j++ ))
do
if [ $i -le $j ]
then
echo -n "*"
else
echo -n " "
if [ $i -eq 7 -a $j -eq 5 ]
then
echo "*"
fi
fi
done
echo ""
done
```

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./pattern.sh
* * * * *
 * * * *
  * * *
   * *
    *
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

PRACTICAL RELATED QUESTIONS :

1) Give output of the following a.

a.

```
#!/bin/sh
```

```
NUMBERS="1 2 3 4 5 6 7"
for NUM in $NUMS
do
Q='expr $NUM % 2
if [ $Q -eq 0 ]
then
echo "Number is an even number!!"
continue
fi
echo "Found odd number"
done
```

Ans.

```
no="1 2 3 4 5 6 7"
```

```
for n in $no
```

```
do
```

```
q=$(( $n % 2 ))
```

```
if test $q -eq 0
```

```
then
```

```
echo "even"
```

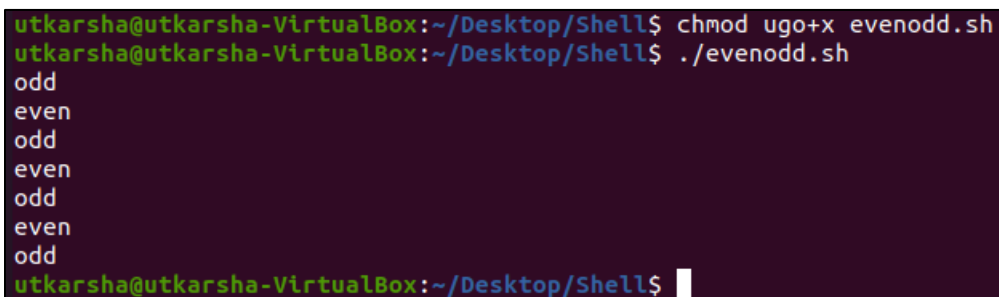
```
continue
```

```
fi
```

```
echo "odd"
```

```
done
```

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x evenodd.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./evenodd.sh
odd
even
odd
even
odd
even
odd
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

b. `#!/bin/sh`

`a=0`

`while [ $a -lt 10 ]`

`do`

`echo $a`

`if [ $a -eq 5 ]`

`then`

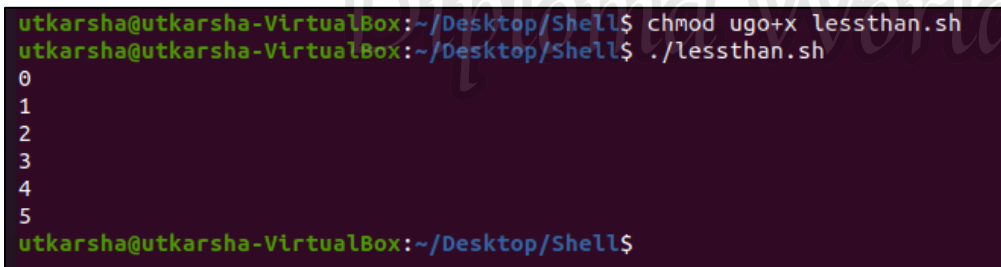
`break`

`fi`

`a=$(( $a + 1 ))`

`done`

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x lessthan.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./lessthan.sh
0
1
2
3
4
5
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

2) State the difference between Iteration and Recursion

Ans.

Property	Recursion	Iteration
Definition	Function calls itself	A set of instructions repeatedly executed
Application	For functions	For loops
Termination	Through base case, where there will be no function call	When the termination condition for the iterator ceases to be satisfied
Usage	Used when code size needs to be small and time complexity is not an issue	Used when time complexity needs to be balanced against an expanded code size

Code Size	Smaller code size	Larger Code Size
Time Complexity	Very high (generally exponential) time complexity	Relatively lower time complexity (generally polynomial-logarithmic)

3) Write a shell script to display Fibonacci series for n numbers.

Ans.

```
read -p 'Enter no for fibonacci = ' no
```

```
x=0
```

```
y=1
```

```
no1=$no
```

```
while [ $no -gt 0 ]
```

```
do
```

```
z=$(( x + y ))
```

```
#echo $z
```

```
x=$y
```

```
y=$z
```

```
no=$(( no - 1 ))
```

```
done
```

```
echo "Fibonacci of $no1 is $y"
```

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./fibonacci.sh
Enter no for Fibonacci = 5
Fibonacci of 5 is 8
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

4) Write a shell script to display tables of 2 to 10 numbers

Ans.

```

utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x table.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./table.sh
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
6 * 1 = 6
6 * 2 = 12
6 * 3 = 18
6 * 4 = 24
6 * 5 = 30
6 * 6 = 36
6 * 7 = 42

```

```

6 * 4 = 24
6 * 5 = 30
6 * 6 = 36
6 * 7 = 42
6 * 8 = 48
6 * 9 = 54
6 * 10 = 60
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80
9 * 1 = 9
9 * 2 = 18
9 * 3 = 27
9 * 4 = 36
9 * 5 = 45
9 * 6 = 54
9 * 7 = 63
9 * 8 = 72
9 * 9 = 81
9 * 10 = 90
10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
10 * 4 = 40
10 * 5 = 50
10 * 6 = 60
10 * 7 = 70
10 * 8 = 80
10 * 9 = 90
10 * 10 = 100
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$

```

5) Write a shell script to accept five digit number and to perform addition of all the digits.

Ans.

```
read -n 5 -p 'Enter 5 digit number: ' no
```

```
sum1=0
```

```
while [ $no -gt 0 ]
```

```
do
```

```
i=$(( no % 10 ))
```

```
sum1=$(( sum1 + i ))
```

```
no=$(( no / 10 ))
```

done

echo " Sum = \$sum1"

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x sum.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./sum.sh
Enter 5 digit number : 12345Sum = 15
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./sum.sh
Enter 5 digit number : 13579 Sum = 25
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

EXERCISE :

Execute the script for the following

- 1) The for loop using day of week list

Ans.

for i in sun mon tue wed thur fri sat

do

echo \$i

done

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./week.sh
sun
mon
tue
wed
thur
fri
sat
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

- 2) The while loop to print different * patterns.

Ans.


```
read -p 'Enter no for pattern = ' no
```

```
i=1
```

```
j=1
```

```
while [ $i -le $no ]
```

```
do
```

```
while [ $j -le $no ]
```

```
do
```

```
if [ $i -le $j ]
```

```
then
```

```
echo -n "*" "
```

```
fi
```

```
j=$(( j + 1 ))
```

```
done
```

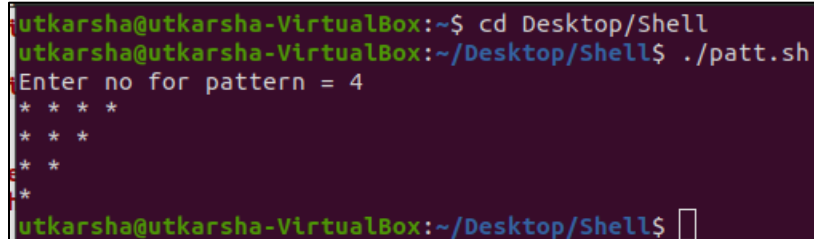
```
j=1
```

```
i=$(( i + 1 ))
```

```
echo ""
```

```
done
```

Output :



```
utkarsha@utkarsha-VirtualBox:~$ cd Desktop/Shell
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./patt.sh
Enter no for pattern = 4
* * * *
* * *
* *
*
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

3) The case statements for performing various mathematical operations.

Ans.

```
read -p "enter 1st no" no1
```

```
read -p "enter 2nd no" no2
```

```
echo -e "1 add \n2 subtract \n3 multiply \n4 divide"
```

```
read -p "enter your choice " ch
```

```
case $ch in
```

```
1) c=$(( $no1 + $no2 ))
```

```
echo "addition = $c ";;
```

```
2) c=$(( $no1 - $no2 ))
```

```
echo "subtraction = $c ";;
```

```
3) c=$(( $no1 * $no2 ))
```

```
echo "multiplication = $c ";;
```

```
4) c=$(( $no1 / $no2 ))
```

```
echo "division = $c ";;
```

```
*) echo "invalid choice";;
```

```
esac
```

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x case.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./case.sh
enter first number : 20
enter second number : 10
1. Add
2. Subtract
3. Multiply
4. Divide
enter choice : 1
addition = 30
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./case.sh
enter first number : 20
enter second number : 10
1. Add
2. Subtract
3. Multiply
4. Divide
enter choice : 2
subtraction = 10
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./case.sh
enter first number : 20
enter second number : 10
1. Add
2. Subtract
3. Multiply
4. Divide
enter choice : 3
multiplication = 200
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./case.sh
enter first number : 20
enter second number : 10
1. Add
2. Subtract
3. Multiply
4. Divide
enter choice : 4
division = 2
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

CONCLUSION : We are able to do different programs using shell script and vi editor.

And also we successfully implemented Shell Scripts.

Practical 12 : Write Shell Script to find out whether – Given file exists?

PRACTICAL SIGNIFICANCE :

In UNIX data and programs are stored in file. These are organized in directories. In a simple way, a directory is just a file that contains other files (or directories). It can be done to find out if a file exists or not on the file system with the help of BASH shell and IF command.

MINIMUM THEORETICAL BACKGROUND :

This can be done by the shell script by using test conditions. Also if..else..fi statement is used which allows us to make decisions based on the success or failure of a command.

Following is the list of the same flags which we can use in file list operators.

Shell

Returns true if

-a file	True if file exists.
-b FILE	True if it is block special.
-c FILE	True if it is character special.
-d file	True if file exists and is a directory.
-e file	True if file exists.
-f file	True if file exists and is a regular file.
-s file	True if file exists and has a size greater than zero.

Example:

```
#!/bin/bash
```

```
echo -e — Enter the name of the file :l
```

```
read file_name
```

```
if [-f file_name]
```

```
then
```

```
else
```

```
echo —$file_name not exists\
```

```
fi
```

Ordinary Files - An ordinary file is a file on the system that contains data text or program instructions. In this tutorial, you look at working with ordinary file.

Directories - directories store both special and ordinary files. For users familiar with Windows or Mac OS Linux directories are equivalent to folder.

Special File - Some special files provide access to hardware such a hard driver, CD ROM driver, modems and Ethernet adapters. Other special files are similar to aliases or shortcuts and enable you to access a single file using different names.

PROGRAM CODE :

Write a shell script which accepts input as a file or directory and displays message whether it is file or directory.

```
echo "Enter file or directory name"
```

```
read file
```

```
if [ -f "$file" ]
```

```
then
```

```
echo "$file It is a file"
```

```
else
```

```
if [ -d "$file" ]
```

```
then
```

```
echo "$file It is a directory"
```

```
fi
```

fi

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ mkdir u1
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ls
case.sh  double.sh  execpwd.sh  f  file.sh  greatest.sh  login.sh  pattern.sh  permission.sh  single.sh  table.sh  week.sh
dir.sh  evenodd.sh  exec.sh  fibo.sh  for.sh  lessthan.sh  multiple.sh  patt.sh  sam.sh  sun.sh  u1
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./dir.sh
Enter file or directory name :
u1
u1 is a directory
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./dir.sh
Enter file or directory name :
multiple.sh
multiple.sh is a file
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

PRACTICAL RELATED QUESTIONS :

1) What is the command to run the script?

Ans. Run the script using `./<filename>`

2) What are the file test options with its meaning?

Ans. The different file test operators are listed below :

- a : True if the file exists
- b : True if it is block special
- c : True if it is a character special file
- d : True if file exists and is a directory
- e : True if the file exists
- f : True if the file exists and is a regular file.
- g : True if the file exists and its SGID bit is set
- h : True if the file exists and is a symbolic link

3) What will be the output of these command (file =ABC.txt)

- a.

```
if [ ! -f —$file ]; then
    echo —$file
fi
```
- b.

```
test -f —$file || echo — $file
```
- c.

```
[ -f —$file ] || echo —$file
```

Ans.

```
osboxes@osboxes:~/Pictures$ ./exists.sh
osboxes@osboxes:~/Pictures$ ./exists1.sh
osboxes@osboxes:~/Pictures$ ./exists2.sh
osboxes@osboxes:~/Pictures$
```

EXERCISE :

1) Write a shell script to copy source file into the destination file.

Ans.

```
echo "Enter File name to copy \c"
```

```
read f1
```

```
echo "Enter file name \c"
```

```
read f2
```

```
if [ -f $f1 ]
```

```
then
```

```
cp $f1 $f2
```

```
else
```

```
echo "$f1 does not exist"
```

```
fi
```

Output :

```
aditi@aditi-VirtualBox:~$ ./aditi.sh
Enter File name to copy \c
a1.sh
Enter file name \c
s1.sh
aditi@aditi-VirtualBox:~$ cat s1.sh
echo "enter value"
read a
echo "$a"
```

2) Write a shell script which displays list of all directories in your home directory.

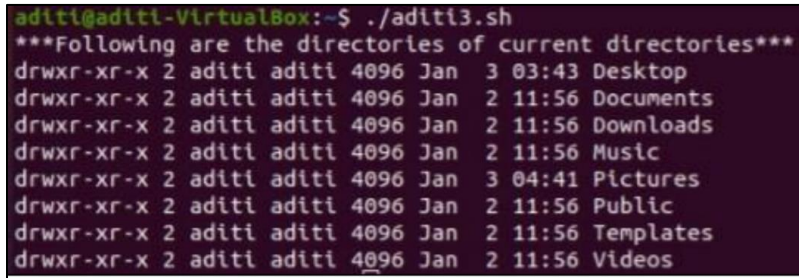
Ans.

```
echo "***Following are the directories of current directories***"
```

```
cd
```

```
ls -l |grep ^d
```

Output :



```
aditi@aditi-VirtualBox:~$ ./aditi3.sh
***Following are the directories of current directories***
drwxr-xr-x 2 aditi aditi 4096 Jan  3 03:43 Desktop
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Documents
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Downloads
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Music
drwxr-xr-x 2 aditi aditi 4096 Jan  3 04:41 Pictures
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Public
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Templates
drwxr-xr-x 2 aditi aditi 4096 Jan  2 11:56 Videos
```

3) Write a shell script which displays list of all files in your home directory.

Ans.

```
echo "Enter directory name"
```

```
read dir
```

```
if [ -d $dir ]
```

```
then
```

```
echo "list of files in the directory"
```

```
cd $dir
```

```
ls
```

```
else
```

```
echo "Enter proper directory name"
```

```
fi
```

Output :


```
aditi@aditi-VirtualBox:~$ ./aditi2.sh
Enter directory name
sayali
Enter proper directory name
aditi@aditi-VirtualBox:~$ ./aditi2.sh
Enter directory name
Documents
list of files in the directory
```

- 4) Write a file handling program. First check whether it is file or directory, then if it is file the program shall ask user for choices of copying, removing and remaining files. Use case statement.

Ans.

```
echo "enter file name"
```

```
read file
```

```
if [ -d "$file" ]
```

```
then
```

```
echo "it is directory"
```

```
else
```

```
echo "Enter your choice"
```

```
echo "1.copy file "
```

```
echo "2.rename file"
```

```
echo "3.remove"
```

```
read ch
```

```
case "$ch" in
```

```
"1") echo "Enter file name to be copied"
```

```
read cpfile
```

```
cp $file $cpfile
```

```
echo "File copied"
```

```
;;
```

```
"2") echo "enter name to rename "
```




```
aditi@aditi-VirtualBox:~$ cat f1.sh
cat: f1.sh: No such file or directory
aditi@aditi-VirtualBox:~$ ./aditi1.sh
enter file name
mn.sh
Enter your choice
1.copy file
2.rename file
3.remove
2
enter name to rename
sample2.sh
mn.sh renamed as sample2.sh
aditi@aditi-VirtualBox:~$ cat mn.sh
cat: mn.sh: No such file or directory
aditi@aditi-VirtualBox:~$ cat sample2.sh
echo "Enter your file name"
aditi@aditi-VirtualBox:~$
```

CONCLUSION : We have successfully completed shell script to find out whether given file exists. In this practical we have studied checking if a file exists, the most commonly used FILE operators are -e and -f

Practical 13 : Write shell script to check and grant permissions

PRACTICAL SIGNIFICANCE :

In Unix files come with permissions, a way to decide who can read, write or execute a file. These permissions are divided into three parts: those for the owner (user) of the file, those for the group to which the owner belongs and then permissions for all the other users from the point of view of what is allowed to do in a file or directory, permissions are for reading, writing and executing.

MINIMUM THEORETICAL BACKGROUND :

Test Commands

Test	Returns true if
-r file	True if file exists and is readable.
-s file	True if file exists and has a size greater than zero.
-w file	True if file exists and is writable.
-x file	True if file exists and is executable.

PROGRAM CODE :

Write a shell script to find out whether - File has read, write and execute permissions.

```
echo "Enter the File : "
```

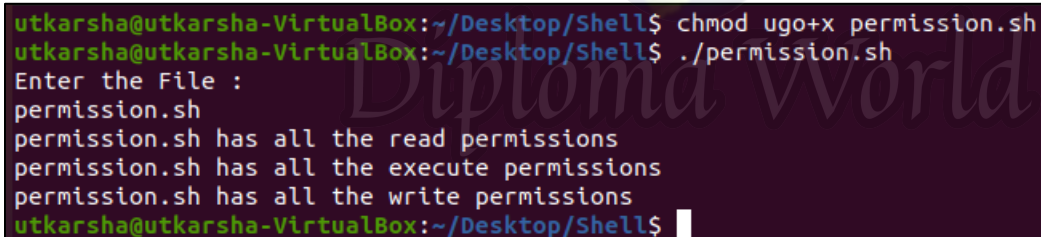
```
read line
```

```
if [ -r $line ]
```

```
then
```

```
echo "$line has all the read permissions"
fi
if [ -x $line ]
then
echo "$line has all the execute permissions"
fi
if [ -w $line ]
then
echo "$line has all the write permissions"
fi
```

Output :



```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x permission.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./permission.sh
Enter the File :
permission.sh
permission.sh has all the read permissions
permission.sh has all the execute permissions
permission.sh has all the write permissions
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

PRACTICAL RELATED QUESTIONS :

1. What are permissions for a file?

Ans. Three permission sets are defined for each file object on a Linux system. These sets include the read, write, and execute permissions for each of three types of users—the owner, the group, and other users.

2. How to assign permission to a file?

Ans. Use the following procedure to change permissions in absolute mode.

- a. If you are not the owner of the file or directory, become super user or assume an equivalent role.
- b. Only the current owner or super user can use the chmod command to change file permissions on a file or directory.
- c. Change permissions in absolute mode by using the chmod command

3. What happens when exception thrown by main() method.

Ans. When exception is thrown by main() method, Java Runtime terminates the program and print the exception message and stack trace in system console. The throws clause only states that the method throws a checked FileNotFoundException and the calling method should catch or rethrow it

4. How to check permissions of all files and directories?

Ans. Check Permissions in Command-Line with Ls Command

If you prefer using the command line, you can easily find a file's permission settings with the ls command, used to list information about files/directories. You can also add the -l option to the command to see the information in the long list

5. What are the test commands to check the permissions of a file?

Ans. The bash shell test command has many more options as follows:

- w FILE : FILE exists and write permission is granted.
- x FILE : FILE exists and execute (or search) permission is granted.
- d FILE : FILE exists and is a directory.
- e FILE : FILE exists.
- f FILE : FILE exists and is a regular file.

EXERCISE :

- 1) Write a shell script which displays the list of all executable files in the current working directory

Ans.

pwd

ls > f

exec < f

while read line

do

if [-f \$line]

then

if [-x \$line]

then

echo "\$line"

fi

fi

done

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x execpwd.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./execpwd.sh
/home/utkarsha/Desktop/Shell
double.sh
execpwd.sh
greatest.sh
login.sh
multiple.sh
permission.sh
sam.sh
single.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

- 2) Write a shell script which displays a list of all the files in the current directory to which user has read, write and execute permission.

Ans.

pwd

ls > f

exec < f

while read line

do

if [-f \$line]

then

if [-r \$line -a -w \$line -a -x \$line]

then

echo "\$line has all the permissions"

fi

fi

done

Output :

```
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x exec.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./exec.sh
/home/utkarsha/Desktop/Shell
double.sh has all the permissions
execpwd.sh has all the permissions
exec.sh has all the permissions
greatest.sh has all the permissions
login.sh has all the permissions
multiple.sh has all the permissions
permission.sh has all the permissions
sam.sh has all the permissions
single.sh has all the permissions
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

3) Write a shell script which accepts a filename and assign it all the permissions.

Ans.

Echo "ENTER THE FILE : "

Read file

Chmod u+xrw \$file

Chmod o+xrw \$file

Chmod g+xrw \$file

Output :

```
utkarsha@utkarsha-VirtualBox:~$ cd Desktop/Shell
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ chmod ugo+x file.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$ ./file.sh
ENTER A FILE :
multiple.sh
utkarsha@utkarsha-VirtualBox:~/Desktop/Shell$
```

CONCLUSION : We have successfully performed scripts to check and grant permission to files.

Practical 14 : Implement Scheduling algorithms

PRACTICAL SIGNIFICANCE :

In Multiprogramming systems, the Operating system schedules the processes on the CPU to have the maximum utilization of it and this procedure is called scheduling. The Operating System uses various scheduling algorithm to schedule the processes. First come first serve (FCFS) scheduling algorithm simply schedules the jobs according their arrival time.

MINIMUM THEORETICAL BACKGROUND :

In Multiprogramming systems, when there is more than one runnable process (i.e., ready), the operating system decides which one to activate. The decision is made by the part of the operating system called the scheduler, using a scheduling algorithm. CPU scheduling selects the processes from memory that are ready to execute, and allocates the CPU to one of them.

PROGRAM CODE :

Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with Arrival Time 0, and given Burst Time, Find using the FCFS scheduling algorithm.

1. Turn Around Time for each process.
2. Waiting Time for each process.
3. Average Turn Around time.
4. Average waiting time.

Process	Burst Time
P1	21

P2	6
P3	3
P4	2

```

#include<stdio.h>
#include<conio.h>
int main()
{
    int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
    printf("\nEnter Total Number of Processes(maximum 20):");
    scanf("\n%d",&n);
    printf("\n enter process burst timen");
    for(i=0;i<n;i++)
    {
        printf("\nP[%d]:",i+1);
        scanf("\n%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time:");
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];
        avwt+=wt[i];
        avtat+=tat[i];
        printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
    }
    avwt/=i;
    avtat/=i;
    printf("\n\nAverage Waitng Time: %d",avwt);
    printf("\n\nAverageTurnaround Time: %d",avtat);
    return 0;
}

```

Result/Output :

```
C:\TURBOC3\BIN>TC

Enter Total Number of Processes(maximum 20):4

enter process burst timen
P11:21

P12:6

P13:3

P14:2

Process      Burst Time    Waiting Time    Turnaround Time:
P11          21            0              21
P12          6            21             27
P13          3            27             30
P14          2            30             32

Average Waiting Time: 19
Average Turnaround Time: 27
```

PRACTICAL RELATED QUESTIONS :

1. Compare SJF, Priority and RR with respect to turnaround time and average waiting time.

Ans.

SJF	RR	Priority
1.Turnaround time is more than FCFS.	1.turnaround time is more than Priority.	1. Turnaround time is less than RR.
2.Average waiting time for given set of processes is minimum.	2.The average waiting time for given set of processes is quite small and depends on the time quantum.	2.In this scheduling algorithm the average waiting time is small as compare to FCFS.

2. State the conditions for preemptive and non-preemptive scheduling algorithm.

Ans.

Conditions for preemptive and non-preemptive scheduling algorithm:-

- When a process switches from the running state to the waiting state(for example, I/O request, or invocation of wait for termination of one of the child processes).
- When a process switches from running state to the ready state (for example, when an interrupt occurs).
- When a process switches from the waiting state to the ready state (for example, completion of I/O).
- When a process terminates.

3. Give the reason of problems arises in FCFS.

Ans. The reason behind this is, in time sharing operating system it is necessary that each user should get CPU share after certain interval. It would be terrific to allow one process to keep the CPU for long time.

4. Write a formula for turnaround Time.

Ans. **Turnaround Time**= Ending Time - Arrival Time

5. Write a formula for Average waiting Time.

Ans. **Average waiting Time** =

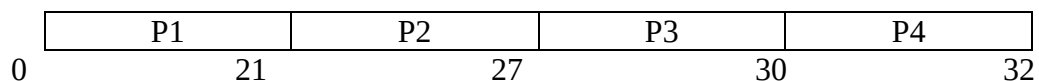
Waiting time of p1+ waiting time of p2+ ...+waiting time of pN / No. of processes.

EXERCISE :

1. Draw the Gantt chart for above mention example.

Process	Burst Time
P1	21
P2	6
P3	3
P4	2

Gantt chart:



2. Calculate average waiting time using RR algorithm for following set of processes with the length of the CPU burst time given in milliseconds. (Time quantum 20 ms).

Process	Burst Time
P1	12
P2	45
P3	78
P4	90

P1	P2	P3	P4	P2	P3	P4	P2	P3	P4	P3	P4	P4
----	----	----	----	----	----	----	----	----	----	----	----	----

0 12 32 52 72 92 112 132 137 157 177 195 215 225

Waiting Time :

P1=0

P2=12+(72-32)+(132-92)=92

P3=32+(92-52)+(137-112)+(177-157)=117

P4=52+(112-72)+(157-132)+(195-177)+(215-215)=135

Average Waiting Time:

$0+92+117+135 \div 4 = 344 \div 4 = 86$

3. The jobs are scheduled for execution are as follows:

Solve the problem using FCFS and Preemptive SJF find average waiting time using Gantt chart.

Process	Arrival Time	Burst Time
P1	0	10
P2	1	4
P3	2	14
P4	3	8

1) FCFS

Gantt Chart:

P1	P2	P3	P4
0	10	14	28
			36

Waiting time

P1=0

$$P2=10-1=9$$

$$P3=14-2=12$$

$$P4=28-3=25$$

$$\text{Average Waiting Time} = 0+9+12+25 \div 4 = 46 \div 4 = 11.5 \text{ msec}$$

2) Pre-emptive SJF:

Gantt chart:

P1	P2	P4	P1	P3	
0	1	5	13	22	36

Waiting Time:

$$P1=12$$

$$P2=1-1=0$$

$$P3=22-2=20$$

$$P4=5-3=2$$

$$\text{Average Waiting Time: } 12+0+20+2 \div 4 = 34 \div 4 = 8.5 \text{ msec}$$

CONCLUSION : We successfully implemented scheduling algorithms to find waiting time and turnaround time with their averages. Also we understood the differences between these algorithms.

Practical 15 : Write a 'C' program to implement FIFO page replacement algorithm

PRACTICAL SIGNIFICANCE :

When there is a page fault, the referenced page must be loaded. If there is no available frame in memory, then one page is selected for replacement. if the selected page has been modified, it must be copied back to disk (swapped out). A page replacement algorithm is needed to decide which page needs to be replaced when a new page comes in.

MINIMUM THEORETICAL BACKGROUND :

This is the simplest page replacement algorithm.in this algorithm operating system keeps track of all pages in the memory in a queue, oldest page is in the front of the queue. when a page needs to be replaced page in the front of the queue is selected for removal.

PROGRAM CODE :

```
#include<stdio.h>

int main()
{
    int i,j,n,a[50],frame[10],no,k, Flag,count=0;

    printf("\n ENTER THE NUMBER OF PAGES:\n");

    scanf("%d",&n);

    printf("\n ENTER THE PAGE NUMBER :\n");

    for(i=1;i<=n;i++)

        scanf("%d",&a[i]);
```

```
printf("\n ENTER THE NUMBER OF FRAMES :");  
scanf("%d",&no);
```

```
for(i=0;i<no;i++)  
    frame[i]= -1;  
    j=0;  
printf("\tref string\t page frames\n");  
for(i=1;i<=n;i++)  
{  
    printf("%d\t\t",a[i]);  
    Flag =0;  
    for(k=0;k<no;k++)  
        if(frame[k]==a[i])  
            Flag=1;  
    if (Flag==0)  
    {  
        frame[j]=a[i];  
        j=(j+1)%no; //inc pos  
        count++;  
        for(k=0;k<no;k++)  
            printf("%d\t",frame[k]);  
    }  
    printf("\n");  
}  
printf("Page Fault Is %d",count);
```

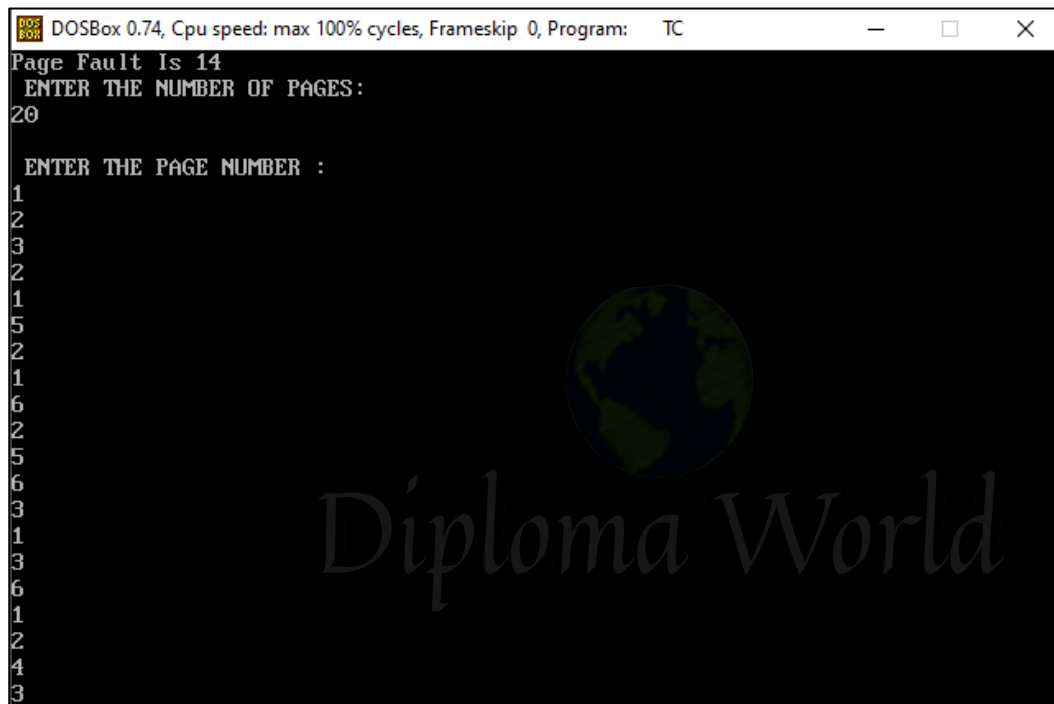


```
        return 0;

        getch();

}
```

Result/Output :

A screenshot of a DOSBox window. The title bar reads "DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC". The window has standard minimize, maximize, and close buttons. The main area is black with white text. The text displayed is: "Page Fault Is 14", "ENTER THE NUMBER OF PAGES:", "20", "ENTER THE PAGE NUMBER :", followed by a vertical list of numbers: 1, 2, 3, 2, 1, 5, 2, 1, 6, 2, 5, 6, 3, 1, 3, 6, 1, 2, 4, 3. A large, semi-transparent watermark is centered in the background, featuring a globe icon and the text "Diploma World".

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Page Fault Is 14
ENTER THE NUMBER OF PAGES:
20
ENTER THE PAGE NUMBER :
1
2
3
2
1
5
2
1
6
2
5
6
3
1
3
6
1
2
4
3
```

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
ENTER THE NUMBER OF FRAMES :3
ref string      page frames
1              1      -1      -1
2              1      2      -1
3              1      2      3
2
1
5              5      2      3
2
1              5      1      3
6              5      1      6
2              2      1      6
5              2      5      6
6
3              2      5      3
1              1      5      3
3
6              1      6      3
1
2              1      6      2
4              4      6      2
3              4      3      2
Page Fault Is 14

```

PRAC
TICAL
RELA
TED
QUES
TIONS
:

1) State the advantages and disadvantages of FIFO.

Ans.

Advantages of FIFO :-

1. FIFO method is easy to understand and operate.
2. FIFO method is useful where transactions are not voluminous and prices of materials are falling.
3. FIFO method is suitable for bulky materials with high unit prices.
4. FIFO method helps to avoid deterioration and obsolescence.
5. Value of closing stock of materials will reflect the current market price.

Disadvantages of FIFO:-

1. FIFO method is improper if many lots are purchased during the period at different prices.
2. The objective of matching current costs with current revenues can not be achieved under FIFO method.
3. If the prices of materials are rising rapidly, the current production cost may be understated.
4. FIFO method overstates profit especially in inflation.

2) What is page hit ?

Ans. If CPU tries to retrieve the needed page from main memory, and that page is existed in the main memory (RAM), then it is known as PAGE HIT.

3) What is page fault ?

Ans. A page fault (PF or hard fault) is a type of exception raised by computer hardware when a running program accesses a memory page that is not currently mapped by the memory management unit (MMU) into the virtual address space of a process.

4) count number of page hits in above mention example given in section x

Ans. Total page hits = 6

EXERCISE :

1) Consider reference string 9,7,6,1,7,6,1,2,7,2 the number of frames in the memory is 3.
Find out number of page faults and page hit respective to

d. Fifo Page replacement algorithm

Ref string	Frames		
9	9		
7	9	7	
6	9	7	6
1	1	7	6
7			
6			
1			
2	1	2	6
7	1	2	7
2			

Total page faults : 6

Total page hits : 4

2) Compare FIFO and LRU.

FIFO	LRU
FIFO I First in First Out	LRU is Least Recently Used

Number of page fault is more than LRU	Number of page fault is less than FIFO
Suffer from belady's anomaly	Does not suffer from belady's anomaly
Simple to implement	Considered to be good

CONCLUSION : We gained knowledge about various page replacement algorithms and successfully implemented FIFO page replacement algorithm.

